

WEBSIFT - The Data Whisperer

A SYNOPSIS

**Submitted in partial fulfillment of the requirement for the award of
Degree of Bachelor of Technology in Artificial Intelligence Machine
Learning (AIML)**

Submitted To



RAJIV GANDHI PROUDYOGIKI VISHWAVIDYALAYA, BHOPAL (M.P.)

Submitted By

Dev Singh Bhatti (0133CL221041)

Jatin Dange (0133CL221060)

Dev Rai (0133CL221040)

Chanchalesh Malvi (0133CL221035)

Under The Supervision of

Dr. Rupali Tabakade

Department of Artificial Intelligence Machine Learning (AIML)



DEPARTMENT OF ARTIFICIAL INTELLIGENCE

SAGAR INSTITUTE OF RESEARCH AND TECHNOLOGY, BHOPAL

SESSION: Jan-June, 2025



Sagar Institute of Research and Technology, Bhopal (M.P.)

CERTIFICATE

This is to certify that the work embodies in this dissertation entitled **“WEBSIFT”** being submitted by-

Dev Singh Bhatti (0133CL221041), Jatin Dange (0133CL221060)

Dev Rai (0133CL221040), Chanchalesh Malvi (0133CL221035)

for partial fulfillment of the requirement for the award of **Bachelor of Technology** in **AIML** discipline to Rajiv Gandhi Pradyogiki Vishwavidyalaya, Bhopal (M.P.) during the academic year 2024-25 is a record of Bonafide piece of work, carried out by them under my supervision and guidance in the **Department of Artificial Intelligence Machine Learning(AIML), Sagar Institute of Research & Technology (SIRT), Bhopal (M.P.)**.

SUPERVISED BY

Dr. Rupali Tabakade

APPROVED BY

**Dr. Kalpana Rai
Professor & Head**

Abstract

This project is a multi-platform web scraper designed to collect product review data from three major Indian e-commerce platforms: Myntra, Flipkart, and Amazon. The system extracts comprehensive review information including product names, prices, ratings, customer reviews, reviewer names, review dates, and overall product ratings. Beyond data collection, the system provides powerful cross-platform comparison capabilities, allowing users to compare prices, ratings, and reviews across all three platforms simultaneously.

The scraper is built in Python with a modular architecture featuring a base scraper class that all platform-specific scrapers inherit from. It utilizes Selenium WebDriver for handling dynamic JavaScript-rendered content and BeautifulSoup for HTML parsing. The system employs parallel scraping using ThreadPoolExecutor to efficiently collect data from multiple platforms concurrently. Data is organized using pandas DataFrames and stored in MongoDB for persistence and querying.

The project includes a Streamlit-based web interface that provides an intuitive user experience. Users can select between single-platform scraping mode or comparison mode, which scrapes all three platforms in parallel and generates comprehensive comparison reports. The comparison feature includes price normalization across different currency formats, rating comparisons, and visual charts displaying price and rating distributions.

This project helps users make informed purchasing decisions by comparing products across multiple e-commerce platforms. It is useful for consumers who want to find the best deals, compare ratings and reviews, or track price trends. The system is also valuable for market researchers studying competitive pricing, product sentiment analysis, and cross-platform review patterns.

ACKNOWLEDGEMENT

The completion of this major project work could be possible with continued & dedicated efforts & guidance of large number of faculty & staff members of the Institute. We acknowledge our gratitude to all of them. The acknowledgement however will be incomplete without specific mention.

We express our profound sense of gratitude to our project coordinator **Dr. Rupali Tabakade, Prof. Vineet Kumar Gupta** for their continuous encouragements & guidance during the project period.

We also express our sincere thanks to the **Dr. Kalpana Rai, HOD, Artificial Intelligence Machine Learning (AIML)** for encouragement & providing all the facilities in the department.

We express our thanks to **Dr. Rajiv Shrivastava (Director), Sagar Institute of Research & Technology (SIRT), Bhopal**, who gave us their valuable support on this project.

Finally, we express our sincere thanks to all the staff members **Department of Artificial Intelligence Machine Learning (AIML) Sagar Institute of Research & Technology (SIRT), Bhopal** for their co-operation in this work and our friends for their timely help co-operation and suggestion to complete this major project work.

Dev Singh Bhatti

(0133CL221041)

Jatin Dange

(0133CL221060)

Dev Rai

(0133CL221040)

Chanchalesh Malvi

(0133CL221035)

B. Tech.-VI Semester

Contents

CERTIFICATE	i
WEBSIFT – “ Scrape.Sort.Succeed”	9
Comprehensive Project Documentation.....	9
1. Project Overview.....	9
1.1 Introduction.....	9
1.2 Core Functionality	9
1.3 Project Goals Data Transformation: Converts raw HTML elements into structured pandas DataFrames with consistent schemas across platforms	9
1.4 Application Scenarios	10
2. About Web Scrapers.....	10
2.1 Definition and Purpose	10
2.2 The Evolution of Web Scraping.....	10
2.3 How Web Scrapers Work: Detailed Process Flow	10
2.4 Static vs Dynamic Content: Technical Challenges	12
2.5 Ethical and Legal Considerations in Web Scraping	12
2.6 Applications of Web Scraping: Detailed Use Cases	13
3. Technologies Used	15
3.1 Python 3.9+ (Anaconda Environment)	15
3.2 Requests Library.....	15
3.3 BeautifulSoup4.....	16
3.4 Fake-UserAgent.....	16
3.5 PyMongo	17
3.6 Dotenv	17
3.7 MongoDB (Local or Atlas)	18
3.9 Selenium WebDriver.....	18
4.1 Project Organization Principles.....	18
4.2 Module Breakdown	19
4.3 Rationale for Modular Design	20
5. Environment Setup (Anaconda).....	20
5.1 Anaconda Overview	20
5.2 Installation Process.....	21
5.3 Package Installation	21
5.4 Environment Management Best Practices.....	22

6. MongoDB Configuration	23
6.1 MongoDB Installation Options	23
6.2 Database Configuration with .env File.....	23
6.3 Database Structure Design	24
7. Implementation: Review Scraping Workflow	26
7.1 Application Entry Point (app.py).....	26
7.2 BaseScraper Architecture Pattern	26
7.3 ComparisonService Implementation.....	27
8. scraper.py	28
8.1 Web Request Management.....	28
8.2 Code Breakdown.....	29
8.3 Future Enhancements.....	30
9. parser.py	32
9.1 HTML Parsing Strategy	32
9.2 Code Breakdown.....	32
9.3 Future Enhancements.....	35
10. db.py.....	36
10.1 Database Integration Strategy.....	36
10.2 Code Breakdown.....	36
10.3 Future Enhancements.....	38
11. requirements.txt.....	40
11.1 Dependency Management Approach	40
11.2 File Contents Breakdown	41
11.3 Future Enhancements.....	42
12. Execution.....	44
12.1 Running the Scraper	44
12.2 Execution Flow Detailed.....	44
12.3 Troubleshooting Common Issues.....	45
12.4 Monitoring and Logging	46
13. Conclusion.....	47
13.1 Project Summary	47
13.2 Architectural Strengths.....	47
13.3 Integration Possibilities.....	48
13.4 Educational Value.....	48

14. Objective	48
14.1 Primary Goals.....	48
14.2 Target Data Fields	49
14.3 Technical Objectives	49
14.4 Learning Objectives.....	49
14.5 Ethical Objectives	49
15. Problem Statement.....	50
15.1 Market Research Challenges.....	50
15.2 Technical Requirements.....	50
15.3 Business Needs	51
15.4 Solution Requirements.....	51
16. Literature Survey.....	52
16.1 Web Scraping Technologies and Approaches.....	52
16.2 Data Storage Solutions for Scraped Content.....	53
16.3 Ethical and Legal Considerations.....	54
16.4 Use Case Studies.....	54
16.5 Technical Architecture Approaches.....	55
17. Proposed Methodology	56
17.1 System Architecture Overview.....	56
17.2 Detailed Process Flow.....	56
17.3 Data Model Design	57
17.4 Error Handling Strategy	58
17.5 Scalability Considerations.....	58
17.6 Ethical Implementation	58
18. Technology Used.....	59
18.1 Programming Language: Python 3.9+	59
18.2 Environment Management: Anaconda.....	59
18.3 HTTP Client: Requests.....	60
18.4 HTML Parser: BeautifulSoup4.....	60
18.5 Database: MongoDB.....	61
18.6 MongoDB Driver: PyMongo.....	61
18.7 Environment Variables: Dotenv	62
18.8 User Agent Rotation: Fake-UserAgent.....	62
19. Result: Multi-Platform Review Scraping and Comparison.....	63

19.1 Scraper Capabilities	63
19.2 Sample Output Analysis.....	64
19.3 Performance Characteristics	64
19.4 Reliability Assessment	65
19.5 Project Success Criteria	65
20. Future Work and Conclusion.....	65
20.1 Future Work.....	65
20.2 Conclusion.....	68
21. References.....	70
21.1 Official Documentation	70
21.2 Web Scraping Resources.....	70
21.3 E-commerce Analysis.....	71
21.4 Project-Specific Resources.....	71

WEBSIFT – “ Scrape.Sort.Succeed”

Comprehensive Project Documentation

By Dev Singh Bhatti, Jatin Dange, Dev Rai, and Chanchalesh Malvi

1. Project Overview

1.1 Introduction

The Multi-Platform Review Scraper project aims to create a robust, scalable Python-based system for extracting product review data from three major Indian e-commerce platforms: Myntra, Flipkart, and Amazon. Unlike many e-commerce websites, these platforms do not provide public APIs for accessing their product review data, making web scraping a necessary approach for comprehensive data collection and cross-platform comparison analysis.

1.2 Core Functionality

Search Query Handling: Accepts a user-provided product search term for querying across multiple platforms (Myntra, Flipkart, Amazon)

1. **Platform Selection:** Supports both single-platform scraping mode and multi-platform comparison mode
2. **Parallel Web Request Management:** Utilizes `ThreadPoolExecutor` to send concurrent HTTP requests to multiple e-commerce platforms simultaneously
3. **Dynamic Content Handling:** Uses Selenium WebDriver to handle JavaScript-rendered content and scroll through review pages
4. **Review Data Extraction:** Parses HTML to extract comprehensive review information including ratings, customer comments, reviewer names, dates, product prices, and overall ratings
5. **Data Normalization:** Normalizes prices across different currency formats and structures from multiple platforms
6. **Cross-Platform Comparison:** Generates comparative analysis including price comparisons, rating comparisons, and review aggregations

1.3 Project Goals **Data Transformation: Converts raw HTML elements into structured pandas DataFrames with consistent schemas across platforms**

Database Integration: Stores the extracted review data in MongoDB collections organized by platform and product name

- **Web Interface:** Provides an intuitive Streamlit-based user interface for interaction and result visualization

- Demonstrating effective use of web scraping techniques with Python Error Handling: Implements robust exception handling with retry mechanisms and graceful degradation when platforms fail
- Implementing proper data storage practices with NoSQL databases
- Building a modular and maintainable codebase
- Ensuring ethical compliance with website terms of service

1.4 Application Scenarios

This scraper can be utilized in various real-world scenarios:

- **Price Monitoring:** Track price changes of specific products over time
- **Competitive Analysis:** Compare pricing strategies across different brands
- **Trend Analysis:** Identify popular products and categories based on availability
- **Stock Monitoring:** Track product availability and restocking patterns
- **Marketing Research:** Gather data for analyzing market positioning and product descriptions

2. About Web Scrapers

2.1 Definition and Purpose

Web scraping refers to the automated process of extracting data from websites. Unlike APIs, which provide structured data through official channels, web scrapers interact with a website's frontend interface, simulating human browsing behavior to collect information that isn't otherwise available in a machine-readable format.

2.2 The Evolution of Web Scraping

The history of web scraping parallels the evolution of the web itself:

1. **Early Days (1990s):** Simple HTTP clients and regular expressions to extract data from static HTML
2. **Web 2.0 Era (2000s):** Development of specialized libraries to handle more complex HTML structures
3. **JavaScript Revolution (2010s):** Introduction of headless browsers to handle dynamic JavaScript content
4. **Modern Era (2020s):** AI-enhanced scraping tools that can understand context and adapt to site changes

2.3 How Web Scrapers Work: Detailed Process Flow

The process of web scraping involves several technical steps:

2.3.1 Request Formation

The scraper first constructs an HTTP request, including:

- The target URL (e.g., `https://www.myntra.com/men-sneakers`)
- Headers that mimic a legitimate browser (User-Agent, Accept, etc.)
- Cookies for maintaining session state (if required)
- Query parameters for search terms, filters, or pagination

In our implementation, the `fetch_search_results()` function in `scraper.py` handles this step, using `requests` and `fake-useragent` libraries to generate authentic-looking HTTP requests.

2.3.2 Content Retrieval

Once the request is sent, the web server processes it and returns an HTTP response containing:

- Status code (200 for success, 404 for not found, etc.)
- Response headers (content type, encoding, etc.)
- The HTML content of the page

Our implementation uses the `requests` library's `.get()` method and validates successful responses with `.raise_for_status()`.

2.3.3 HTML Parsing

The raw HTML must be parsed into a navigable structure. This typically involves:

- Converting the HTML string into a Document Object Model (DOM)
- Creating a tree-like representation of the HTML elements
- Providing methods to search and traverse this tree

We use BeautifulSoup with the “html.parser” engine to create this structured representation of the HTML.

2.3.4 Data Extraction

With the parsed HTML, the scraper can then:

- Locate specific elements using selectors (CSS, XPath, etc.)
- Extract text, attributes, and nested content
- Clean and normalize the extracted data

In our project, the `parse_product_cards()` function identifies product elements using class selectors like `.product-base` and extracts specific data points like brand, name, and price.

2.3.5 Data Storage

Finally, the extracted data is:

- Structured into a consistent format (JSON, dictionaries, etc.)
- Validated for completeness and accuracy

- Saved to persistent storage (database, files, etc.)

We use MongoDB as our storage solution, structuring each product as a document within a collection.

2.4 Static vs Dynamic Content: Technical Challenges

2.4.1 Static Content

Static web pages deliver the same HTML content with each server response. Key characteristics include:

- Complete HTML delivered directly from the server
- Content visible in the page source
- Can be scraped with simple HTTP requests and HTML parsing
- Examples include basic blogs, documentation sites, and older websites

Scraping static content is relatively straightforward since all the required data is present in the initial HTML response.

2.4.2 Dynamic Content

Modern websites often employ JavaScript frameworks to render content after the initial page load:

- Initial HTML serves as a shell or template
- Content is loaded asynchronously via API calls
- Rendered in the browser using JavaScript
- Not visible in the page source, only in the rendered DOM
- Examples include single-page applications built with React, Angular, or Vue

Scraping dynamic content presents additional challenges:

- Simple HTTP clients cannot see the fully rendered content
- Requires browser automation or JavaScript execution
- Must wait for asynchronous content to load
- May need to interact with elements to trigger data loading

Myntra uses a mix of static and dynamic content. While some product information is available in the initial HTML, certain elements (like real-time discounts or stock status) may be loaded dynamically. A full-featured scraper would need capabilities to handle both types of content.

2.5 Ethical and Legal Considerations in Web Scraping

Web scraping exists in a complex ethical and legal landscape that varies by jurisdiction and website policy.

2.5.1 Key Legal Considerations

- **Terms of Service:** Most websites explicitly prohibit or restrict automated data collection
- **Rate Limiting:** Excessive requests may constitute a denial-of-service attack
- **Copyright Law:** Extracted content may be protected by copyright
- **Data Protection Regulations:** Scraping personal data may violate GDPR, CCPA, or similar laws
- **Computer Fraud and Abuse Acts:** Unauthorized access to protected systems may be illegal

2.5.2 Ethical Best Practices

To conduct web scraping responsibly:

1. **Respect robots.txt:** Check and honor the website's crawling directives
2. **Implement rate limiting:** Space out requests to avoid server overload
3. **Identify your scraper:** Use a descriptive User-Agent that allows site owners to contact you
4. **Cache results:** Minimize duplicate requests by storing and reusing data
5. **Scrape only what you need:** Limit collection to the specific data required for your use case
6. **Consider alternative sources:** Use official APIs when available

2.5.3 Technical Implementation of Ethical Scraping

Our Myntra scraper incorporates several ethical practices:

1. **Random User-Agent rotation:** Using fake-useragent to distribute requests across different browser signatures
2. **Error handling with backoff:** Retrying failed requests with increasing delays
3. **Session management:** Reusing HTTP sessions to reduce connection overhead
4. **Data validation:** Ensuring we only store complete and accurate product records

2.6 Applications of Web Scraping: Detailed Use Cases

2.6.1 Price Tracking and Comparison

E-commerce price monitoring involves:

- Collecting price data across multiple retailers
- Tracking historical price changes
- Identifying sales patterns and discounting strategies
- Alerting consumers to price drops

Services like CamelCamelCamel (for Amazon) use web scraping to track price histories and provide alerts to consumers.

2.6.2 Real Estate Market Analysis

Real estate platforms aggregate listings from multiple sources:

- Property details (size, bedrooms, amenities)
- Price history and current asking prices
- Location data and neighborhood statistics
- Market trends and valuation estimates

Zillow, Redfin, and similar platforms employ sophisticated scraping systems to build comprehensive property databases.

2.6.3 News and Content Aggregation

Content aggregators compile information from diverse sources:

- Headlines and article summaries
- Publication dates and authors
- Topic classification and categorization
- Sentiment analysis of content

Google News and similar services use scraping techniques to collect and organize content from thousands of news sources.

2.6.4 Job Market Monitoring

Recruitment platforms and job market analyzers use scraping to:

- Collect job postings across company websites and job boards
- Track salary ranges and compensation packages
- Identify in-demand skills and qualifications
- Monitor hiring trends by industry and location

Indeed, Glassdoor, and LinkedIn use these techniques to build comprehensive job databases.

2.6.5 Academic and Research Applications

Research institutions use web scraping for:

- Literature reviews and metadata collection
- Dataset compilation from public sources
- Tracking citations and research impact
- Monitoring publication patterns in academic journals

Tools like Google Scholar and Scopus employ scraping techniques to index academic content.

3. Technologies Used

3.1 Python 3.9+ (Anaconda Environment)

Python serves as the foundation of our scraping project due to its robust ecosystem of data processing libraries and straightforward syntax.

3.1.1 Why Python for Web Scraping?

Python offers several advantages for web scraping projects:

- **Rich Library Ecosystem:** Extensive libraries specifically designed for web requests, HTML parsing, and data processing
- **Readability:** Clean syntax that makes scraping logic easy to understand and maintain
- **Cross-Platform:** Works consistently across Windows, macOS, and Linux
- **Community Support:** Large community with abundant documentation and examples
- **Data Science Integration:** Seamless connection to data analysis tools like pandas and matplotlib

3.1.2 Anaconda Distribution Benefits

We chose Anaconda as our Python distribution for several reasons:

- **Environment Management:** Isolated environments prevent dependency conflicts
- **Package Handling:** Simplified installation of complex packages with binary dependencies
- **Cross-Platform Consistency:** Ensures consistent behavior across different operating systems
- **Pre-built Packages:** Optimized binaries for scientific and data processing libraries
- **Environment Sharing:** Easily reproducible environments through environment.yml files

3.2 Requests Library

The Requests library handles HTTP communication with the Myntra website.

3.2.1 Key Features Used

- **Session Management:** Maintains cookies and connection pooling for efficiency
- **Header Customization:** Allows setting User-Agent and other HTTP headers to mimic browser behavior
- **Response Handling:** Provides methods for processing various response formats
- **Error Management:** Built-in exception handling for HTTP errors
- **Timeout Configuration:** Controls connection and read timeouts to prevent hanging

3.2.2 Implementation Details

In our scraper, Requests is used to:

- Construct properly formatted URLs with search parameters
- Set appropriate headers to avoid detection
- Handle response status codes and exceptions
- Retrieve the HTML content for parsing

3.3 BeautifulSoup4

BeautifulSoup4 is our HTML parsing library of choice.

3.3.1 Parsing Capabilities

- **Multiple Parser Backends:** Compatible with “html.parser”, “lxml”, and “html5lib”
- **Navigable DOM:** Provides intuitive navigation through HTML element trees
- **Powerful Selectors:** Supports CSS selectors, tag names, attributes, and more
- **Text Extraction:** Methods for retrieving clean text from elements
- **Attribute Access:** Simple syntax for accessing element attributes

3.3.2 Implementation in Our Project

We use BeautifulSoup to:

- Parse the HTML response into a structured DOM
- Locate product card elements using class selectors
- Extract specific data points from each card
- Navigate through nested elements to find prices, names, etc.

3.4 Fake-UserAgent

This utility library helps our scraper avoid detection by rotating through realistic browser user-agent strings.

3.4.1 Importance of User-Agent Rotation

Many websites implement anti-scraping measures that:

- Block IPs that use non-browser User-Agents
- Identify patterns of identical User-Agents in rapid succession
- Serve different content to different browser types

Fake-UserAgent helps circumvent these measures by:

- Providing realistic, up-to-date User-Agent strings
- Rotating between different browser types (Chrome, Firefox, Safari, etc.)
- Matching version numbers to current browser releases

3.4.2 Implementation in Our Scraper

We integrate Fake-UserAgent in the `fetch_search_results()` function to:

- Generate a random, realistic User-Agent for each request
- Distribute requests across different simulated browser types
- Reduce the likelihood of being flagged as a bot

3.5 PyMongo

PyMongo is the official MongoDB driver for Python, providing the interface between our scraper and the database.

3.5.1 Core Functionality

- **Connection Management:** Handles database connections and authentication
- **CRUD Operations:** Provides methods for creating, reading, updating, and deleting documents
- **Query Building:** Supports MongoDB's query language for complex data retrieval
- **Cursor Handling:** Efficiently manages result sets from database queries
- **Index Management:** Creates and optimizes indexes for performance

3.5.2 Implementation in Our Project

We use PyMongo to:

- Establish a connection to the MongoDB server
- Access the appropriate database and collection
- Insert extracted product data as documents
- Implement batch operations for efficiency
- Handle potential database exceptions

3.6 Dotenv

The Dotenv library allows us to securely manage configuration variables.

3.6.1 Configuration Management Approach

- **Environment Separation:** Different settings for development, testing, and production
- **Sensitive Data Protection:** Keeps credentials out of version control
- **Simplified Configuration:** Centralizes settings in a single file
- **Portability:** Makes the application easily deployable across different environments

3.6.2 Implementation in Our Project

We use Dotenv to manage:

- MongoDB connection strings
- Database and collection names

- Any API keys or credentials (if needed for future extensions)
- Configuration toggles for scraper behavior

3.7 MongoDB (Local or Atlas)

MongoDB serves as our data storage solution due to its flexibility with semi-structured data.

3.7.1 Why MongoDB for Web Scraping

MongoDB offers several advantages for storing scraped data:

- **Schema Flexibility:** Accommodates varying data structures from different product types
- **Document Model:** Natural fit for hierarchical product data
- **Query Capabilities:** Powerful query language for data analysis
- **Indexing:** Efficient retrieval of products by various attributes
- **Scalability:** Handles growing datasets through sharding and replication

3.7.2 Local vs. Atlas Deployment

Our project supports both deployment options:

Local MongoDB:

- Faster development iterations
- No network latency
- Complete control over configuration
- Suitable for development and testing

MongoDB Atlas:

- Cloud-hosted solution
- Automatic backups and security updates
- Scalable resources
- 3.11 Concurrent Execution: ThreadPoolExecutor

3.9 Selenium WebDriver

4.1 Project Organization Principles

Our directory structure follows these design principles:

- **Separation of Concerns:** Different aspects of functionality in separate modules
- **Single Responsibility:** Each module handles one specific part of the scraping process
- **Clear Dependencies:** Explicit imports show relationships between modules
- **Configuration Isolation:** Environment variables separated from code

- **Documentation Inclusion:** README and requirements files for proper project documentation

4.2 Module Breakdown

4.2.1 *main.py*

The entry point and orchestrator of the scraping process:

- Imports functionality from other modules
- Coordinates the overall scraping workflow
- Handles command-line arguments (in future extensions)
- Provides status reporting and logging

4.2.2 *scraper.py*

Responsible for HTTP interaction with the target website:

- Constructs proper URLs for searches
- Manages HTTP requests and headers
- Handles network errors and retries
- Returns raw HTML content

4.2.3 *parser.py*

Handles the extraction of data from HTML:

- Parses HTML using BeautifulSoup
- Locates product elements in the page structure
- Extracts specific data points from each product
- Transforms raw data into structured format

4.2.4 *db.py*

Manages database operations:

- Establishes connection to MongoDB
- Creates and selects appropriate databases and collections
- Implements data storage operations
- Handles database errors

4.2.5 *.env*

Stores configuration variables:

- Database connection strings
- Collection names
- Other environment-specific settings

4.2.6 requirements.txt

Lists project dependencies:

- Specific package names
- Version requirements
- Used for reproducing the environment

4.2.7 README.md

Project documentation:

- Installation instructions
- Usage examples
- Design explanations
- Contribution guidelines

4.3 Rationale for Modular Design

This structure provides several advantages:

- **Maintainability:** Changes to one aspect (e.g., parsing logic) don't affect others
- **Testability:** Individual components can be tested in isolation
- **Extensibility:** New features can be added by extending specific modules
- **Reusability:** Components like the database module can be reused in other projects
- **Collaboration:** Team members can work on different modules simultaneously

5. Environment Setup (Anaconda)

5.1 Anaconda Overview

5.1.1 What is Anaconda?

Anaconda is a distribution of Python and R programming languages for scientific computing, data science, and machine learning. It includes:

- The Python interpreter
- Over 1,500 pre-built and tested packages
- The conda package and environment manager
- Development tools like Jupyter and Spyder
- Utilities for data visualization and analysis

5.1.2 Advantages of Anaconda for Web Scraping Projects

Anaconda provides several benefits specifically relevant to web scraping:

- **Dependency Resolution:** Automatically handles complex package dependencies
- **Binary Packages:** Pre-compiled libraries that work across platforms
- **Environment Isolation:** Prevents conflicts between different project requirements

- **Reproducibility:** Environment export/import for consistent setup across machines
- **Cross-Platform:** Works consistently on Windows, macOS, and Linux

5.2 Installation Process

5.2.1 Anaconda Installation

Before setting up the project environment, Anaconda must be installed:

1. Download the appropriate Anaconda installer for your platform from the official website
2. Run the installer and follow the prompts
3. Verify installation by opening Anaconda Navigator or running `conda --version` in a terminal

5.2.2 Environment Creation Command Explanation

The command to create our project environment is:

```
conda create -n myntra_scraper python=3.9
```

This command:

- Invokes the conda environment manager
- Creates a new environment named “myntra_scraper”
- Specifies Python 3.9 as the interpreter version
- Sets up a clean, isolated environment

5.2.3 Environment Activation

After creation, the environment must be activated:

```
conda activate myntra_scraper
```

This command:

- Switches the current shell to use the myntra_scraper environment
- Updates PATH variables to use packages from this environment
- Changes the command prompt to indicate the active environment

5.3 Package Installation

5.3.1 Package Installation Command Breakdown

```
pip install requests beautifulsoup4 fake-useragent pymongo python-dotenv
```

This command:

- Uses pip (the Python package installer) within the conda environment
- Installs the five main dependencies required by our project
- Pulls the latest compatible version of each package

5.3.2 Why Both Conda and Pip?

While conda is the primary package manager for Anaconda, we use pip in this case because:

- Some packages (like fake-useragent) are more up-to-date on PyPI than in conda channels
- Pip installations are sometimes simpler for pure Python packages
- Our dependencies don't have complex binary requirements that benefit from conda's binary distribution system

5.3.3 Alternative: Using a conda-forge Channel

For a pure conda approach, the alternative would be:

```
conda install -c conda-forge requests beautifulsoup4 pymongo python-dotenv
conda install -c conda-forge pip
pip install fake-useragent
```

This uses conda for packages available in the conda-forge channel and falls back to pip only for packages not available through conda.

5.4 Environment Management Best Practices

5.4.1 Environment Sharing

To make the environment reproducible, use:

```
conda env export > environment.yml
```

This exports all dependencies (including exact versions) to an environment.yml file that can be shared with collaborators.

5.4.2 Environment Recreation

To recreate the environment on another machine:

```
conda env create -f environment.yml
```

This creates an identical environment with all the same packages and versions.

5.4.3 Environment Update

When new dependencies are needed:

1. Activate the environment: `conda activate myntra_scraper`
2. Install new packages: `pip install new-package`
3. Update the requirements.txt: `pip freeze > requirements.txt`

5.4.4 Environment Cleanup

When the environment is no longer needed:

```
conda deactivate
conda env remove -n myntra_scraper
```

This removes the environment and frees up disk space.

6. MongoDB Configuration

6.1 MongoDB Installation Options

6.1.1 Local MongoDB Installation

For development and testing, a local MongoDB installation provides:

- Lower latency for database operations
- Offline development capability
- Complete control over configuration
- No usage limits or costs

Installation steps vary by platform:

- **Windows:** Download and run the MongoDB Community Server installer
- **macOS:** Install via Homebrew with `brew install mongodb-community`
- **Linux:** Follow distribution-specific package installation (apt, yum, etc.)

6.1.2 MongoDB Atlas Setup

For production or team-based development:

1. Create an account on MongoDB Atlas
2. Create a new cluster (free tier available)
3. Configure network access (IP whitelisting)
4. Create database user credentials
5. Obtain the connection string

Atlas provides:

- Managed infrastructure
- Automatic backups
- Scaling capabilities
- Global distribution options

6.2 Database Configuration with .env File

6.2.1 Purpose of Environment Variables

Using environment variables for configuration:

- Keeps sensitive information out of source code
- Allows different configurations for different environments

- Simplifies deployment across multiple systems
- Follows security best practices

6.2.2 .env File Structure Explained

```
MONGO_URI=mongodb://localhost:27017/
DB_NAME=myntra_db
COLLECTION_NAME=products
```

These variables define:

- **MONGO_URI:** The connection string for MongoDB (local or Atlas)
- **DB_NAME:** The database name to store product data
- **COLLECTION_NAME:** The collection name within the database

6.2.3 Connection String Variations

Local MongoDB Connection:

```
MONGO_URI=mongodb://localhost:27017/
```

- Uses default host and port
- No authentication required for development setup

Atlas Connection with Authentication:

```
MONGO_URI=mongodb+srv://username:password@cluster0.mongodb.net/?retryWrites=true&w=majority
```

- Includes username and password
- Specifies the cluster hostname
- Includes connection options

6.2.4 Security Considerations

To protect sensitive information:

- Add .env to .gitignore to prevent committing credentials
- Use different .env files for development, testing, and production
- Consider using a secrets management system for production deployments
- Limit database user permissions to only what's necessary

6.3 Database Structure Design

6.3.1 Database Organization

Our MongoDB structure uses:

- A single database (myntra_db) for all Myntra data
- Separate collections for different data types
- Initially, just a products collection for basic product data

6.3.2 Collection Schema Design

While MongoDB is schema-less, we implement a consistent structure:

```
{
  "brand": "String - The product brand name",
  "name": "String - The product name/title",
  "price": "Number - The current price in INR",
  "url": "String - The full product page URL",
  "image_url": "String - URL to the product image",
  "scraped_at": "Date - When the product was scraped",
  "category": "String - Product category (future enhancement)",
  "discounted_from": "Number - Original price if discounted (future enhancement)",
  "rating": "Number - Product rating (future enhancement)"
}
```

6.3.3 Indexing Strategy

For optimal query performance, we create indexes on frequently queried fields:

```
# To be added to db.py in future enhancement
collection.create_index("brand")
collection.create_index("price")
collection.create_index([("brand", 1), ("price", 1)]) # Compound index
collection.create_index("scraped_at") # For time-based queries
```

6.3.4 Data Validation

MongoDB provides schema validation capabilities, which we could add for data integrity:

```
db.createCollection("products", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["brand", "name", "price", "url", "image_url", "scraped_at"],
      properties: {
        brand: { bsonType: "string" },
        name: { bsonType: "string" },
        price: { bsonType: "int" },
        url: { bsonType: "string" },
        image_url: { bsonType: "string" },
        scraped_at: { bsonType: "date" }
      }
    }
  }
})
```

7. Implementation: Review Scraping Workflow

7.1 Application Entry Point (app.py)

7.1.1 Streamlit Application Structure

The app.py module serves as the Streamlit web application entry point and orchestrates the entire scraping workflow:

- User Interface: Provides web-based forms for product search and platform selection
- Mode Selection: Offers two modes - Single Platform scraping or Compare All Platforms
- Session Management: Maintains user state using Streamlit session_state
- Scraper Instantiation: Dynamically creates appropriate scraper instances based on user selection
- Result Display: Renders scraped data and comparison results in interactive format

Database Integration: Stores scraped reviews in MongoDB via MongoIO class

- 7.1.2 Single Platform Mode Workflow
- When user selects Single Platform mode:
 1. User enters product name and selects platform (Myntra, Flipkart, or Amazon)
 2. System instantiates the corresponding scraper class (ScrapeReviews, FlipkartScraper, or AmazonScraper)

7.2 BaseScraper Architecture Pattern

7.2.1 Abstract Base Class Design

The BaseScraper class (in src/scrapper/base_scraper.py) provides a common foundation:

```
from abc import ABC, abstractmethod
```

- from selenium import webdriver
-
- class BaseScraper(ABC):

```
def __init__(self, product_name: str, no_of_products: int):
```

```
    # Common setup: WebDriver configuration, Chrome options
```

```
    # Anti-bot detection measures
```

- # Wait strategies
-
- @abstractmethod
- def scrape_product_urls(self, product_name: str) -> list:

- """Platform-specific product URL extraction"""
- pass

@abstractmethod

def extract_reviews(self, product_link) -> object:

- """Navigate to product page and extract review links/metadata"""
- pass
-

7.3 ComparisonService Implementation

7.3.1 Command-Line Interface

A more flexible version would accept parameters:

```
import argparse
```

```
def parse_arguments():
    parser = argparse.ArgumentParser(description='Myntra product scraper')
    parser.add_argument('search_term', help='Product search term')
    parser.add_argument('--pages', type=int, default=1, help='Number of pages
to scrape')
    parser.add_argument('--output', choices=['db', 'csv', 'json'], default='d
b',
                        help='Output format')
    return parser.parse_args()
```

```
def main():
    args = parse_arguments()
    html = fetch_search_results(args.search_term)
    # Rest of the function...
```

7.3.2 Configuration Management

Adding configuration from environment or files:

```
import os
from dotenv import load_dotenv

def load_config():
    load_dotenv()
    return {
        'search_term': os.getenv('SEARCH_TERM', 'men sneakers'),
        'pages': int(os.getenv('PAGES', '1')),
        'output_format': os.getenv('OUTPUT_FORMAT', 'db')
    }
```

```
def main():
    config = load_config()
    # Use config values instead of hardcoded values
```

7.3.3 Logging Implementation

Adding proper logging:

```
import logging

def setup_logging():
    logging.basicConfig(
        level=logging.INFO,
        format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
        filename='myntra_scraper.log'
    )
    return logging.getLogger('myntra_scraper')

def main():
    logger = setup_logging()
    logger.info(f"Starting scraper with search term: {search_term}")
    # Rest of function with logging instead of print statements
```

7.3.4 Error Handling

More robust error handling:

```
def main():
    try:
        # Existing code
    except Exception as e:
        logger.error(f"Scraping failed: {e}")
        return 1 # Non-zero exit code for failure
    return 0 # Success
```

8. scraper.py

8.1 Web Request Management

8.1.1 The Role of scraper.py

The scraper.py module is responsible for:

- Constructing properly formatted URLs for Myntra searches
- Setting appropriate headers to mimic browser behavior
- Sending HTTP requests to Myntra's servers
- Handling response status and errors
- Returning the raw HTML content for further processing

8.1.2 Separation from Parsing Logic

We deliberately separate HTTP requests from HTML parsing to:

- Keep each module focused on a single responsibility
- Allow for easier testing of each component
- Enable future enhancements (like adding caching) without affecting other parts
- Make the code more maintainable and understandable

8.2 Code Breakdown

import requests

from fake_useragent **import** UserAgent

```
def fetch_search_results(query):  
    headers = {"User-Agent": UserAgent().random}  
    url = f"https://www.myntra.com/{query.replace(' ', '-')}"  
    response = requests.get(url, headers=headers)  
    response.raise_for_status()  
    return response.text
```

8.2.1 Import Statements

import requests

from fake_useragent **import** UserAgent

These imports bring in:

- The requests library for HTTP communication
- The UserAgent class from fake-useragent for generating realistic browser headers

8.2.2 Function Definition

```
def fetch_search_results(query):
```

The function:

- Takes a single parameter query (the search term)
- Returns the HTML content as a string
- Has a clear, descriptive name indicating its purpose

8.2.3 Header Configuration

```
headers = {"User-Agent": UserAgent().random}
```

This line:

- Creates a dictionary of HTTP headers
- Uses UserAgent().random to generate a different, realistic User-Agent string for each request
- Helps avoid detection as a bot by mimicking normal browser behavior

8.2.4 URL Construction

```
url = f"https://www.myntra.com/{query.replace(' ', '-')}"
```

This line:

- Uses an f-string for clear URL formatting
- Replaces spaces in the query with hyphens to match Myntra's URL structure
- Creates the complete URL for the search results page

8.2.5 HTTP Request

```
response = requests.get(url, headers=headers)
```

This line:

- Makes a GET request to the constructed URL
- Passes the headers dictionary to set request headers
- Stores the response object for further processing

8.2.6 Error Handling

```
response.raise_for_status()
```

This method:

- Checks if the response status code indicates an error (4xx or 5xx)
- Raises an HTTPError exception if the request failed
- Ensures we only process successful responses

8.2.7 Result Return

```
return response.text
```

This line:

- Extracts the response body as a text string
- Returns the raw HTML content for parsing by other components

8.3 Future Enhancements

8.3.1 Session Management

For better performance and cookie handling:

```
def create_session():  
    session = requests.Session()  
    session.headers.update({"User-Agent": UserAgent().random})  
    return session  
  
def fetch_search_results(query, session=None):  
    if session is None:  
        session = create_session()  
    url = f"https://www.myntra.com/{query.replace(' ', '-')}"
```

```

response = session.get(url)
response.raise_for_status()
return response.text

```

8.3.2 Retry Logic

For handling temporary failures:

```

from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

def create_session_with_retries():
    session = requests.Session()
    retry = Retry(
        total=3,
        backoff_factor=0.5,
        status_forcelist=[500, 502, 503, 504]
    )
    adapter = HTTPAdapter(max_retries=retry)
    session.mount('http://', adapter)
    session.mount('https://', adapter)
    session.headers.update({"User-Agent": UserAgent().random})
    return session

```

8.3.3 Pagination Support

To scrape multiple pages of results:

```

def fetch_search_results_paged(query, pages=1):
    all_html = []
    session = create_session()
    for page in range(1, pages + 1):
        url = f"https://www.myntra.com/{query.replace(' ', '-')}"
        if page > 1:
            url += f"?p={page}"
        response = session.get(url)
        response.raise_for_status()
        all_html.append(response.text)
    return all_html

```

8.3.4 Proxy Support

For distributing requests across different IP addresses:

```

def fetch_search_results(query, proxy=None):
    headers = {"User-Agent": UserAgent().random}
    proxies = {"http": proxy, "https": proxy} if proxy else None
    url = f"https://www.myntra.com/{query.replace(' ', '-')}"
    response = requests.get(url, headers=headers, proxies=proxies)
    response.raise_for_status()
    return response.text

```


8.3.5 Rate Limiting

To avoid overloading the target server:

```
import time

def fetch_search_results(query, delay=1.0):
    headers = {"User-Agent": UserAgent().random}
    url = f"https://www.myntra.com/{query.replace(' ', '-')}"
    time.sleep(delay) # Wait before making request
    response = requests.get(url, headers=headers)
    response.raise_for_status()
    return response.text
```

9. parser.py

9.1 HTML Parsing Strategy

9.1.1 The Role of parser.py

The parser.py module is responsible for:

- Parsing raw HTML into a navigable structure
- Locating product elements within the page
- Extracting specific data points from each product
- Transforming unstructured HTML into structured data objects
- Handling parsing errors and exceptions

9.1.2 BeautifulSoup Utilization

BeautifulSoup provides several key capabilities:

- Converting HTML strings into navigable DOM trees
- Locating elements using various selector methods
- Extracting text and attributes from elements
- Navigating parent-child relationships in the DOM
- Handling malformed HTML gracefully

9.2 Code Breakdown

```
from bs4 import BeautifulSoup

def parse_product_cards(html):
    soup = BeautifulSoup(html, "html.parser")
    product_list = []
    cards = soup.find_all("li", class_="product-base")
    for card in cards:
        try:
            brand = card.find("h3", class_="product-brand").text.strip()
            name = card.find("h4", class_="product-product").text.strip()
```

```

        price_tag = card.find("span", class_="product-discountedPrice")
        price = price_tag.text.replace("Rs. ", "") if price_tag else None
        link = "https://www.myntra.com" + card.find("a")["href"]
        image_url = card.find("img")["src"]
        product_list.append({
            "brand": brand,
            "name": name,
            "price": int(price) if price else None,
            "url": link,
            "image_url": image_url
        })
    except Exception as e:
        print(f"Error parsing card: {e}")
    return product_list

```

9.2.1 Import Statement

```
from bs4 import BeautifulSoup
```

This imports the BeautifulSoup class, which is the primary tool for HTML parsing.

9.2.2 Function Definition

```
def parse_product_cards(html):
```

The function:

- Takes raw HTML content as input
- Returns a list of product dictionaries
- Has a descriptive name indicating its purpose

9.2.3 Soup Creation

```
soup = BeautifulSoup(html, "html.parser")
```

This line:

- Creates a BeautifulSoup object from the HTML string
- Uses the built-in “html.parser” for parsing
- Creates a navigable representation of the HTML DOM

9.2.4 Result Container

```
product_list = []
```

Initializes an empty list to store the extracted product data.

9.2.5 Finding Product Cards

```
cards = soup.find_all("li", class_="product-base")
```

This line:

- Uses the find_all method to locate all product cards
- Targets elements with the class “product-base”

- Returns a list of matching elements

9.2.6 Iterating Through Cards

for card **in** cards:

This begins a loop to process each product card individually.

9.2.7 Error Handling

```
try:
    # Extraction code
except Exception as e:
    print(f"Error parsing card: {e}")
```

This structure:

- Wraps the extraction code in a try-except block
- Catches any exceptions that occur during parsing
- Logs the error message
- Continues processing other cards even if one fails

9.2.8 Data Extraction

```
brand = card.find("h3", class_="product-brand").text.strip()
name = card.find("h4", class_="product-product").text.strip()
price_tag = card.find("span", class_="product-discountedPrice")
price = price_tag.text.replace("Rs. ", "") if price_tag else None
link = "https://www.mynta.com" + card.find("a")["href"]
image_url = card.find("img")["src"]
```

These lines:

- Find specific elements within each card using tag names and classes
- Extract text content using the .text property
- Clean the text using .strip() to remove whitespace
- Handle conditional elements (like price_tag) that might not exist
- Extract attributes using dictionary syntax (e.g., ["href"], ["src"])
- Transform data as needed (removing "Rs." prefix from prices)

9.2.9 Data Structure Creation

```
product_list.append({
    "brand": brand,
    "name": name,
    "price": int(price) if price else None,
    "url": link,
    "image_url": image_url
})
```

This code:

- Creates a dictionary with standardized keys for each product

- Converts the price to an integer if available
- Adds the complete product dictionary to the result list

9.2.10 Result Return

```
return product_list
```

Returns the complete list of products after processing all cards.

9.3 Future Enhancements

9.3.1 Additional Data Fields

Extracting more product details:

```
# Inside the try block
discount = card.find("span", class_="product-discountPercentage")
discount_percent = discount.text.strip() if discount else None

sizes_available = card.find("h4", class_="product-sizes")
sizes = sizes_available.text.strip() if sizes_available else None

# Add to dictionary
product_list.append({
    # Existing fields...
    "discount_percent": discount_percent,
    "sizes": sizes
})
```

9.3.2 Data Validation

Implementing basic validation:

```
def is_valid_product(product):
    """Check if a product has all required fields with valid values."""
    required_fields = ["brand", "name", "price", "url", "image_url"]
    return all(product.get(field) for field in required_fields)

# In the parsing function
if is_valid_product(product_dict):
    product_list.append(product_dict)
else:
    print(f"Skipping invalid product: {product_dict}")
```

9.3.3 Structured Logging

Replacing print statements with proper logging:

```
import logging
logger = logging.getLogger(__name__)

# Replace print statements with logger calls
```

```
except Exception as e:
    logger.error(f"Error parsing card: {e}", exc_info=True)
```

9.3.4 Alternative Parser Engines

Using a different parser backend for better performance:

```
# Try to use lxml for faster parsing
try:
    soup = BeautifulSoup(html, "lxml")
except ImportError:
    # Fall back to built-in parser if lxml is not available
    soup = BeautifulSoup(html, "html.parser")
```

9.3.5 CSS Selector Support

Using CSS selectors for more concise element selection:

```
brand = card.select_one(".product-brand").text.strip()
name = card.select_one(".product-product").text.strip()
price_tag = card.select_one(".product-discountedPrice")
```

10. db.py

10.1 Database Integration Strategy

10.1.1 The Role of db.py

The db.py module handles all MongoDB interactions:

- Loading database configuration from environment variables
- Establishing connections to the MongoDB server
- Selecting the appropriate database and collection
- Implementing data storage operations
- Handling database errors and exceptions

10.1.2 PyMongo Implementation

PyMongo provides the Python interface to MongoDB:

- Creating client connections with appropriate options
- Accessing databases and collections
- Performing CRUD operations (Create, Read, Update, Delete)
- Handling MongoDB-specific errors

10.2 Code Breakdown

```
from pymongo import MongoClient
from dotenv import load_dotenv
import os
```

```
load_dotenv()

MONGO_URI = os.getenv("MONGO_URI")
DB_NAME = os.getenv("DB_NAME")
COLLECTION_NAME = os.getenv("COLLECTION_NAME")

client = MongoClient(MONGO_URI)
db = client[DB_NAME]
collection = db[COLLECTION_NAME]

def save_to_mongodb(data):
    if isinstance(data, list):
        collection.insert_many(data)
    else:
        collection.insert_one(data)
```

10.2.1 Import Statements

```
from pymongo import MongoClient
from dotenv import load_dotenv
import os
```

These imports bring in:

- MongoClient class for MongoDB connections
- load_dotenv function to load environment variables
- os module to access environment variables

10.2.2 Environment Loading

```
load_dotenv()
```

This loads environment variables from a .env file into the process environment.

10.2.3 Configuration Variables

```
MONGO_URI = os.getenv("MONGO_URI")
DB_NAME = os.getenv("DB_NAME")
COLLECTION_NAME = os.getenv("COLLECTION_NAME")
```

These lines:

- Retrieve configuration values from environment variables
- Store them in module-level constants
- Make the configuration accessible throughout the module

10.2.4 Database Connection

```
client = MongoClient(MONGO_URI)
db = client[DB_NAME]
collection = db[COLLECTION_NAME]
```

This code:

- Creates a MongoDB client connection using the URI
- Selects the target database
- Selects the target collection
- Establishes these connections at module import time

10.2.5 Save Function

```
def save_to_mongodb(data):
    if isinstance(data, list):
        collection.insert_many(data)
    else:
        collection.insert_one(data)
```

This function:

- Takes data to be saved (single document or list of documents)
- Checks the data type using `isinstance()`
- Uses the appropriate insert method based on data type
- Handles both single and batch insert operations

10.3 Future Enhancements

10.3.1 Connection Pooling and Lazy Loading

Improving connection management:

```
# Module-level client but lazy collection access
_client = None
```

```
def get_client():
    global _client
    if _client is None:
        load_dotenv()
        _client = MongoClient(os.getenv("MONGO_URI"))
    return _client
```

```
def get_collection():
    client = get_client()
    db = client[os.getenv("DB_NAME")]
    return db[os.getenv("COLLECTION_NAME")]
```

```
def save_to_mongodb(data):
    collection = get_collection()
    # Rest of the function
```

10.3.2 Error Handling

Adding robust error handling:

```
from pymongo.errors import ConnectionFailure, OperationFailure
```

```

def save_to_mongodb(data):
    try:
        if isinstance(data, list):
            result = collection.insert_many(data)
            return len(result.inserted_ids)
        else:
            result = collection.insert_one(data)
            return result.inserted_id
    except ConnectionFailure as e:
        logger.error(f"Database connection failed: {e}")
        raise
    except OperationFailure as e:
        logger.error(f"Database operation failed: {e}")
        raise

```

10.3.3 Data Deduplication

Preventing duplicate entries:

```

def save_to_mongodb(data, allow_duplicates=False):
    if not allow_duplicates:
        if isinstance(data, list):
            result = []
            for item in data:
                update_result = collection.update_one(
                    {"url": item["url"]},
                    {"$set": item},
                    upsert=True
                )
                result.append(update_result)
            return result
        else:
            return collection.update_one(
                {"url": data["url"]},
                {"$set": data},
                upsert=True
            )
    else:
        # Original insert code

```

10.3.4 Timestamp Addition

Adding timestamps to entries:

```

from datetime import datetime

def save_to_mongodb(data):
    current_time = datetime.utcnow()

    if isinstance(data, list):
        for item in data:

```



```

        item["scraped_at"] = current_time
    collection.insert_many(data)
else:
    data["scraped_at"] = current_time
    collection.insert_one(data)

```

10.3.5 Query Methods

Adding data retrieval functions:

```

def find_products(query=None, limit=10):
    """Find products matching the query."""
    if query is None:
        query = {}
    return list(collection.find(query).limit(limit))

def find_product_by_url(url):
    """Find a specific product by its URL."""
    return collection.find_one({"url": url})

def count_products(query=None):
    """Count products matching the query."""
    if query is None:
        query = {}
    return collection.count_documents(query)

```

11. requirements.txt

11.1 Dependency Management Approach

11.1.1 Purpose of requirements.txt

The requirements.txt file serves several important purposes:

- Documents all project dependencies in one place
- Enables easy installation of dependencies with `pip install -r requirements.txt`
- Ensures consistent environments across different systems
- Functions as a form of documentation for the project's technical stack
- Simplifies onboarding new developers to the project

11.1.2 Versioning Strategies

There are several approaches to dependency versioning:

1. **Loose Versioning:** Just listing package names without versions

```

requests
beautifulsoup4

```

2. **Pinned Versions:** Specifying exact versions

```
requests==2.28.1
beautifulsoup4==4.11.1
```

3. **Minimum Versions:** Specifying minimum compatible versions

```
requests>=2.28.1
beautifulsoup4>=4.11.1
```

4. **Version Ranges:** Specifying ranges of compatible versions

```
requests>=2.28.1,<3.0.0
beautifulsoup4>=4.11.1,<5.0.0
```

Our project currently uses loose versioning, which offers flexibility but may lead to compatibility issues if dependencies release breaking changes.

11.2 File Contents Breakdown

```
requests
beautifulsoup4
fake-useragent
pymongo
python-dotenv
```

11.2.1 *requests*

The HTTP library for making web requests:

- Handles GET, POST, and other HTTP methods
- Manages cookies, headers, and redirects
- Provides response status and content handling
- Essential for communicating with the Myntra website

11.2.2 *beautifulsoup4*

The HTML parsing library:

- Converts HTML strings into navigable DOM structures
- Provides methods for locating and extracting elements
- Handles malformed HTML gracefully
- Core component for extracting structured data from HTML

11.2.3 *fake-useragent*

A utility for generating realistic User-Agent strings:

- Provides a database of current browser User-Agent strings
- Helps avoid detection as a scraper bot
- Rotates between different browser identities
- Reduces the likelihood of being blocked by the website

11.2.4 pymongo

The official MongoDB driver for Python:

- Manages connections to MongoDB servers
- Provides CRUD operations for documents
- Handles MongoDB-specific data types and operations
- Essential for storing the scraped data

11.2.5 python-dotenv

A utility for loading environment variables:

- Reads .env files and loads variables into the environment
- Simplifies configuration management
- Keeps sensitive data out of source code
- Enables different configurations for different environments

11.3 Future Enhancements

11.3.1 Version Pinning

For improved reproducibility, pin specific versions:

```
requests==2.28.1
beautifulsoup4==4.11.1
fake-useragent==1.1.1
pymongo==4.3.3
python-dotenv==1.0.0
```

11.3.2 Development Dependencies

Separate development dependencies:

```
# Runtime Dependencies
requests==2.28.1
beautifulsoup4==4.11.1
fake-useragent==1.1.1
pymongo==4.3.3
python-dotenv==1.0.0

# Development Dependencies
pytest==7.3.1
flake8==6.0.0
black==23.3.0
```

11.3.3 Optional Dependencies

Mark optional components:

```
# Core Dependencies
requests==2.28.1
beautifulsoup4==4.11.1
pymongo==4.3.3
python-dotenv==1.0.0

# Optional - User Agent Rotation
fake-useragent==1.1.1

# Optional - Performance Improvements
lxml==4.9.2 # Faster HTML parsing
```

11.3.4 Dependency Groups

Using pip's dependency groups feature:

```
# Core
requests==2.28.1
beautifulsoup4==4.11.1
pymongo==4.3.3
python-dotenv==1.0.0

# Advanced Features
selenium==4.9.0 # For JavaScript rendering
pandas==2.0.0   # For data analysis

# Development
pytest==7.3.1
black==23.3.0
```

11.3.5 Constraints File

Separate the direct dependencies from transitive constraints:

requirements.in (direct dependencies):

```
requests
beautifulsoup4
fake-useragent
pymongo
python-dotenv
```

requirements.txt (generated with pip-compile):

```
certifi==2022.12.7
    # via requests
charset-normalizer==3.1.0
    # via requests
fake-useragent==1.1.1
    # via -r requirements.in
idna==3.4
```

```
# via requests
pymongo==4.3.3
# via -r requirements.in
python-dotenv==1.0.0
# via -r requirements.in
requests==2.28.1
# via -r requirements.in
soupsieve==2.4.1
# via beautifulsoup4
urllib3==1.26.15
# via requests
```

12. Execution

12.1 Running the Scraper

12.1.1 Command Explanation

```
python main.py
```

This command:

- Invokes the Python interpreter
- Executes the main.py script as the entry point
- Requires that the Anaconda environment is activated
- Assumes all dependencies are installed
- Uses the current working directory for relative paths

12.1.2 Execution Environment

For proper execution, ensure:

- The Anaconda environment is activated: `conda activate myntra_scraper`
- MongoDB is running (if using a local installation)
- The .env file is properly configured
- All dependencies are installed: `pip install -r requirements.txt`
- The working directory contains all project files

12.2 Execution Flow Detailed

12.2.1 Initialization Phase

1. Python interpreter starts and imports the main module
2. Module imports are resolved (scraper.py, parser.py, db.py)
3. Database connections are established (in db.py)
4. The main() function is called

12.2.2 Request Phase

1. The search term is used to construct a Myntra search URL
2. A random User-Agent is generated

3. An HTTP GET request is sent to Myntra
4. The response is validated and its content extracted

12.2.3 Parsing Phase

1. The HTML content is passed to BeautifulSoup
2. Product card elements are located in the HTML
3. Data is extracted from each product card
4. Product dictionaries are created with standardized fields
5. A list of product dictionaries is assembled

12.2.4 Storage Phase

1. The product list is passed to the database module
2. MongoDB insert operations are performed
3. Success or failure status is determined
4. Results are reported to the console

12.3 Troubleshooting Common Issues

12.3.1 Environment Issues

Problem: ModuleNotFoundError: No module named 'requests' **Solution:** Ensure the environment is activated and dependencies installed:

```
conda activate myntra_scraper
pip install -r requirements.txt
```

12.3.2 Database Connection Issues

Problem: pymongo.errors.ServerSelectionTimeoutError: No servers found

Solution: Verify MongoDB is running and the connection string is correct:

```
# Check if MongoDB is running
mongosh
```

```
# Verify .env file
cat .env
```

12.3.3 Scraping Failures

Problem: requests.exceptions.HTTPError: 403 Client Error: Forbidden **Solution:** The website may be blocking scraping attempts:

- Add more realistic headers
- Implement request delays
- Use a proxy service
- Reduce request frequency

12.3.4 Parsing Issues

Problem: `AttributeError: 'NoneType' object has no attribute 'text'` **Solution:** The HTML structure may have changed:

- Inspect the current HTML structure
- Update selectors in `parser.py`
- Add more robust error handling
- Implement fallback extraction methods

12.3.5 Performance Issues

Problem: Slow execution or high memory usage **Solution:**

- Implement pagination to process results in batches
- Use a more efficient HTML parser (lxml)
- Optimize database operations with bulk inserts
- Implement caching for repeated requests

12.4 Monitoring and Logging

12.4.1 Basic Console Output

The current implementation provides simple console output:

25 products saved to MongoDB.

12.4.2 Enhanced Logging Implementation

For better visibility, implement proper logging:

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler("myntra_scraper.log"),
        logging.StreamHandler()
    ]
)

logger = logging.getLogger(__name__)

def main():
    # Replace print statements with logger calls
    logger.info(f"Starting scraper with search term: {search_term}")
    # ...
    logger.info(f"{len(products)} products saved to MongoDB.")
```

13. Conclusion

13.1 Project Summary

The Myntra Web Scraper project successfully demonstrates:

- Effective web scraping techniques with Python
- Modular software design principles
- NoSQL database integration with MongoDB
- Environment management with Anaconda
- Ethical scraping practices and considerations

This implementation provides a flexible foundation for extracting product data from Myntra.com, with a clean separation of concerns that makes the code maintainable and extensible.

13.2 Architectural Strengths

13.2.1 Modularity

The project demonstrates strong modularity through:

- Separation of HTTP requests (scraper.py) from HTML parsing (parser.py)
- Isolation of database operations (db.py) from business logic
- Clear entry point and orchestration (main.py)
- Well-defined interfaces between components

This approach makes the codebase:

- Easier to understand and maintain
- More testable (components can be tested in isolation)
- More adaptable to future changes

13.2.2 Configuration Management

The use of environment variables and .env files provides:

- Secure handling of sensitive information (connection strings)
- Easy deployment across different environments
- Separation of configuration from code
- Flexibility to change database settings without code modifications

13.2.3 Error Handling

The implementation includes basic error handling:

- Try-except blocks around parsing operations
- Status validation for HTTP responses
- Graceful handling of missing data elements

- Reporting of both success and failure states

13.3 Integration Possibilities

This project can be extended to integrate with:

13.3.1 Data Analysis Platforms

- Connect with pandas for data manipulation and analysis
- Export data to CSV/Excel for further processing
- Feed data into Jupyter notebooks for interactive analysis
- Integrate with visualization tools like Tableau or Power BI

13.3.2 Monitoring and Alerting Systems

- Implement price drop alerts via email or SMS
- Track inventory changes and notify when items become available
- Monitor discount percentages and identify sales events
- Create dashboards for real-time market analysis

13.3.3 Business Intelligence Applications

- Feed data into BI tools for trend analysis
- Track competitor pricing strategies
- Monitor brand presence and product offerings
- Analyze seasonal patterns in fashion retail

13.4 Educational Value

This project serves as an educational resource for:

- Python development best practices
- Web scraping techniques and ethics
- NoSQL database design and usage
- Environment management with Anaconda
- Modular software architecture principles

By examining and extending this code, developers can gain practical experience with these technologies in a realistic application context.

14. Objective

14.1 Primary Goals

The primary objective of this project is to develop a comprehensive Python-based web scraping system that:

1. **Automates Data Collection:** Eliminates manual data gathering from Myntra.com
2. **Structures Raw Data:** Transforms unstructured HTML into usable data formats
3. **Preserves Information:** Stores extracted data for analysis and reference

4. **Maintains Flexibility:** Adapts to different search queries and product categories
5. **Ensures Scalability:** Handles varying amounts of data efficiently
6. **Promotes Reproducibility:** Uses standardized environments and dependencies

14.2 Target Data Fields

The scraper focuses on extracting these key product attributes:

- **Brand Name:** The manufacturer or fashion label
- **Product Name:** The specific item name or description
- **Current Price:** The discounted or selling price
- **Product URL:** The direct link to the product page
- **Image URL:** The link to the product's primary image

14.3 Technical Objectives

From a technical perspective, the project aims to:

1. **Demonstrate Clean Architecture:** Separate concerns into distinct modules
2. **Apply Python Best Practices:** Follow PEP 8 style guidelines and idiomatic Python
3. **Implement Error Handling:** Gracefully manage exceptions and edge cases
4. **Enable Configurability:** Use environment variables for flexible configuration
5. **Ensure Reproducibility:** Leverage Anaconda for consistent environments
6. **Facilitate Extensibility:** Design for future enhancements and features

14.4 Learning Objectives

As an educational project, it also serves to:

1. **Illustrate Web Scraping Techniques:** Show practical approaches to data extraction
2. **Demonstrate MongoDB Usage:** Showcase NoSQL database integration
3. **Highlight Environment Management:** Present Anaconda as a solution for dependency control
4. **Explore HTTP Interactions:** Demonstrate proper usage of the requests library
5. **Practice HTML Parsing:** Demonstrate effective use of BeautifulSoup

14.5 Ethical Objectives

The project emphasizes ethical scraping by:

1. **Respecting Website Resources:** Implementing measures to prevent server overload
2. **Following Terms of Service:** Acknowledging the importance of website policies
3. **Using User-Agent Rotation:** Mimicking normal browser behavior
4. **Implementing Rate Limiting:** Spacing requests to minimize impact

5. **Collecting Only Necessary Data:** Focusing on publicly available product information

15. Problem Statement

15.1 Market Research Challenges

15.1.1 Scale and Complexity of E-commerce Data

Modern e-commerce platforms like Myntra represent vast, dynamic data ecosystems:

- **Product Volume:** Myntra hosts tens of thousands of products across numerous categories
- **Frequent Updates:** Product listings, prices, and availability change constantly
- **Seasonal Variations:** Fashion inventory shifts with seasons and trends
- **Flash Sales:** Limited-time discounts and promotions occur regularly
- **Category Diversity:** Products span clothing, footwear, accessories, and home goods

15.1.2 Manual Data Collection Limitations

Attempting to track this information manually presents severe limitations:

- **Time Constraints:** Manual collection is prohibitively time-consuming
- **Human Error:** Manual recording introduces transcription errors
- **Inconsistency:** Different collectors may use different formats or standards
- **Limited Coverage:** Only a small sample of products can be realistically tracked
- **Delayed Insights:** Data becomes outdated before analysis can be completed

15.1.3 Lack of Official Data Access

Unlike some platforms, Myntra does not provide public data access methods:

- **No Public API:** There is no official programmatic interface for product data
- **Limited Export Options:** No bulk download capabilities for research purposes
- **Restricted Data Usage:** Terms of service may limit commercial use of data
- **No Structured Feeds:** No RSS or other structured data feeds available

15.2 Technical Requirements

To address these challenges, a technical solution must:

15.2.1 Automated Data Extraction

- **Search Functionality:** Ability to extract data based on user-defined search terms
- **Consistent Retrieval:** Reliable extraction of data across different product types
- **Scheduled Execution:** Capability to run at specified intervals (future enhancement)
- **Error Recovery:** Ability to continue operation despite occasional failures

15.2.2 Dynamic Content Handling

- **HTML Parsing:** Effective extraction of data from complex HTML structures
- **Element Location:** Reliable identification of product elements within pages
- **Attribute Extraction:** Accurate retrieval of product attributes and metadata
- **Format Normalization:** Consistent data formatting across different product categories

15.2.3 Structured Data Storage

- **Schema Design:** Appropriate data model for product information
- **Query Capability:** Ability to retrieve and filter stored data
- **Data Integrity:** Prevention of duplicate or corrupted entries
- **Scalability:** Capacity to handle growing data volumes over time

15.2.4 Ethical Compliance

- **Rate Limiting:** Prevention of excessive requests that might impact site performance
- **Identification:** Proper identification of the scraper through User-Agent strings
- **Respect for TOS:** Adherence to website terms of service
- **Resource Efficiency:** Minimal impact on the target website's infrastructure

15.3 Business Needs

The solution addresses several specific business needs:

15.3.1 Market Intelligence

- **Competitor Pricing:** Understanding how different brands price similar products
- **Discount Patterns:** Identifying sales patterns and promotion strategies
- **Product Range Analysis:** Comparing product offerings across brands and categories
- **Market Positioning:** Analyzing how brands position themselves in the market

15.3.2 Trend Identification

- **Popular Products:** Identifying frequently restocked or prominently featured items
- **Seasonal Patterns:** Recognizing seasonal trends in fashion inventory
- **Price Fluctuations:** Tracking how prices change over time
- **Brand Presence:** Monitoring which brands dominate specific categories

15.3.3 Decision Support

- **Pricing Strategy:** Informing pricing decisions with competitive data
- **Product Development:** Identifying gaps or opportunities in the market
- **Inventory Planning:** Guiding stocking decisions based on market trends
- **Marketing Focus:** Determining which products or categories to emphasize

15.4 Solution Requirements

Based on these challenges and needs, the ideal solution should:

- **Automatically extract** product data from search results
- **Handle changes** in website structure and dynamic content
- **Store data** in a structured, queryable format (NoSQL database)
- **Operate ethically**, respecting website limitations
- **Provide flexibility** to extract data from different product categories
- **Enable analysis** through well-structured data storage
- **Maintain reproducibility** through standardized environments

16. Literature Survey

16.1 Web Scraping Technologies and Approaches

16.1.1 HTML Parsing Libraries

Research in web scraping has identified several effective libraries for HTML parsing:

BeautifulSoup:

- Widely adopted for its intuitive API and forgiving HTML parsing
- Supports multiple parser backends (html.parser, lxml, html5lib)
- Excellent for navigating and searching HTML document trees
- Somewhat slower than lxml but more user-friendly

lxml:

- C-based implementation offering higher performance
- Supports both HTML and XML with XPath capabilities
- More strict about HTML correctness than BeautifulSoup
- Better suited for performance-critical applications

Scrapy:

- Full-featured web crawling framework
- Includes built-in support for handling requests, parsing, and data pipelines
- Offers more robust and scalable architecture for large projects
- Has a steeper learning curve but provides more capabilities

16.1.2 HTTP Request Handling

Studies on effective web scraping emphasize the importance of request management:

Requests Library:

- Simple, human-friendly API for HTTP operations
- Supports cookies, sessions, and authentication
- Handles most common HTTP interactions effectively
- Industry standard for Python HTTP clients

aiohttp:

- Asynchronous HTTP client for parallel requests
- Significantly improves performance for I/O-bound scraping tasks
- Requires asynchronous programming patterns
- Better suited for high-volume scraping operations

selenium/playwright:

- Browser automation tools for JavaScript-heavy sites
- Handle dynamic content that simple HTTP clients cannot access
- Simulate real user interactions with websites
- More resource-intensive but necessary for some modern websites

16.2 Data Storage Solutions for Scraped Content

Research into data storage options for web scraping highlights several approaches:

16.2.1 NoSQL Databases

MongoDB:

- Document-oriented structure matches web content organization
- Schema flexibility accommodates varying product structures
- Good performance for write-heavy workloads
- Native JSON-like document format aligns with web data

Elasticsearch:

- Powerful full-text search capabilities
- Good for applications needing complex querying of scraped text
- Structured document storage with flexible mappings
- Higher resource requirements than simpler databases

Redis:

- In-memory data store with optional persistence
- Extremely fast for caching and temporary storage
- Limited complex query capabilities
- Well-suited for rate limiting and job queues in scraping architectures

16.2.2 Relational Databases

PostgreSQL/MySQL:

- Strong data integrity guarantees
- Powerful query capabilities with SQL
- Requires more upfront schema design

- Better for highly structured data with consistent fields

SQLite:

- Lightweight, file-based database
- No separate server process required
- Limited concurrency support
- Good for smaller scraping projects or prototypes

16.3 Ethical and Legal Considerations

Research and industry publications emphasize several important ethical considerations:

16.3.1 Legal Framework

The legal landscape for web scraping varies by jurisdiction but typically involves:

- **Computer Fraud and Abuse Act (US):** Prohibits unauthorized access to protected computer systems
- **GDPR (EU):** Regulates the collection and processing of personal data
- **Copyright Law:** Protects original content from unauthorized reproduction
- **Terms of Service:** May explicitly prohibit automated data collection
- **Trespass to Chattels:** Legal theory applied to server resource consumption

Notable cases like *hiQ Labs v. LinkedIn* and *Ryanair v. PR Aviation* have shaped the legal understanding of web scraping boundaries.

16.3.2 Best Practices

Industry consensus on ethical scraping includes:

- **robots.txt Compliance:** Respecting the Robots Exclusion Protocol
- **Rate Limiting:** Implementing reasonable request frequencies
- **Identification:** Using informative User-Agent strings
- **Terms of Service Review:** Understanding and respecting website policies
- **Minimal Impact:** Designing scrapers to reduce server load
- **Data Minimization:** Collecting only necessary information

16.4 Use Case Studies

Research literature highlights several successful applications of web scraping in e-commerce:

16.4.1 E-commerce Analytics

Price Monitoring Systems:

- Track competitor pricing across multiple platforms
- Identify pricing strategies and discount patterns

- Inform dynamic pricing algorithms
- Alert to significant price changes

Inventory Analysis:

- Monitor stock levels and availability
- Identify popular products by restocking frequency
- Track seasonal product introductions
- Measure product lifecycle stages

16.4.2 Market Research Applications

Brand Presence Analysis:

- Measure relative market share by product counts
- Track brand positioning and category focus
- Identify brand entry into new categories
- Monitor brand collaboration patterns

Consumer Trend Identification:

- Analyze product descriptions for emerging keywords
- Track aesthetic trends through image analysis
- Identify price point migrations over time
- Monitor sustainability and ethical claims in product marketing

16.5 Technical Architecture Approaches

Research on scraping system design highlights several architectural patterns:

16.5.1 Modular Design

- Separation of concerns between HTTP requests, parsing, and storage
- Pluggable components for different website structures
- Centralized configuration management
- Clear interfaces between system components

16.5.2 Distributed Scraping

- Work distribution across multiple machines
- IP rotation through proxy services
- Queue-based task management
- Centralized result aggregation and deduplication

16.5.3 Resilience Patterns

- Backoff and retry mechanisms for transient failures
- Circuit breakers for persistent errors
- Checkpointing for resumable operations
- Monitoring and alerting for system health

17. Proposed Methodology

17.1 System Architecture Overview

The Myntra Web Scraper follows a modular, pipeline-based architecture that separates different aspects of the scraping process:

1. **Input Layer:** Defines search parameters and configuration
2. **Request Layer:** Handles HTTP communication with Myntra
3. **Parsing Layer:** Extracts structured data from HTML responses
4. **Storage Layer:** Persists data to MongoDB
5. **Output Layer:** Reports operation results

This separation provides clear boundaries between components, making the system easier to maintain, test, and extend.

17.2 Detailed Process Flow

17.2.1 Input Processing

The scraping process begins with a search term provided by the user:

1. The search term is processed to conform to Myntra's URL structure
2. Any special characters are properly encoded
3. The term is converted to lowercase and spaces are replaced with hyphens
4. This becomes the basis for constructing the search URL

17.2.2 HTTP Request Construction

The system constructs an appropriate HTTP request:

1. A full URL is formed using Myntra's base URL and the processed search term
2. Headers are prepared, including a randomized User-Agent string
3. The request is configured with appropriate timeout values
4. The GET request is sent to Myntra's servers

17.2.3 Response Handling

Upon receiving the HTTP response:

1. The status code is checked to ensure successful retrieval
2. If unsuccessful, appropriate error handling is triggered
3. For successful responses, the HTML content is extracted
4. The content is passed to the parsing layer

17.2.4 HTML Parsing

The HTML content is processed to extract product information:

1. The HTML is parsed into a navigable DOM structure using BeautifulSoup
2. Product card elements are located using class selectors
3. For each product card:
 - Brand name is extracted from the h3.product-brand element
 - Product name is extracted from the h4.product-product element
 - Price is extracted from the span.product-discountedPrice element
 - Product URL is constructed from the anchor href attribute
 - Image URL is extracted from the img src attribute
4. All extracted data is formatted into a standardized dictionary structure

17.2.5 Data Transformation

The raw extracted data undergoes transformation:

1. Text fields are stripped of extra whitespace
2. Price values have currency symbols removed
3. Numeric fields are converted to appropriate types (e.g., price to integer)
4. URLs are made absolute by adding base URLs where needed
5. The data is structured into a consistent dictionary format

17.2.6 Database Storage

The transformed data is persisted to MongoDB:

1. A connection to the MongoDB server is established
2. The appropriate database and collection are selected
3. Data is validated for required fields and formats
4. For batch operations, insert_many() is used for efficiency
5. For single items, insert_one() is used
6. The operation results are captured for reporting

17.2.7 Result Reporting

The system provides feedback on the operation:

1. The number of products successfully stored is counted
2. A summary message is generated with operation statistics
3. This information is displayed to the user via console output
4. In future enhancements, this could include logging to files or external systems

17.3 Data Model Design

The product data model follows this structure:

```
{  
  "brand": "String - The manufacturer or brand name",  
  "name": "String - The specific product name",  
}
```

```
"price": "Integer - The current price in INR",  
"url": "String - Full URL to the product page",  
"image_url": "String - URL to the product image",  
"scraped_at": "Date - When this data was collected (future enhancement)"  
}
```

This model:

- Captures the essential product information
- Uses appropriate data types for each field
- Provides sufficient information for basic analysis
- Can be extended with additional fields in future versions

17.4 Error Handling Strategy

The system implements several layers of error handling:

1. **Network Errors:** HTTP request failures are caught and reported
2. **Parsing Errors:** Exceptions during HTML parsing are caught per product
3. **Data Validation:** Missing or malformed data is handled gracefully
4. **Database Errors:** Failures in storage operations are captured and reported

This multi-layered approach ensures that:

- Partial failures don't halt the entire process
- As much data as possible is recovered even when some elements fail
- Users receive meaningful error messages
- The system can be debugged effectively when issues occur

17.5 Scalability Considerations

The current design supports several scalability paths:

1. **Pagination:** Future versions can iterate through multiple result pages
2. **Category Expansion:** The approach can be extended to different product categories
3. **Parallel Processing:** Multiple searches could be executed concurrently
4. **Scheduled Execution:** The system could run on a regular schedule to track changes
5. **Data Versioning:** Historical data could be preserved to track trends over time

These considerations have influenced the initial design decisions, ensuring that the foundation supports future growth and enhancement.

17.6 Ethical Implementation

The methodology includes specific measures for ethical scraping:

1. **User-Agent Rotation:** Using fake-useragent to distribute requests
2. **Request Spacing:** Future enhancements will include deliberate delays between requests

3. **Minimal Data Collection:** Only collecting publicly available product information
4. **Efficient Parsing:** Extracting only needed data to minimize resource usage
5. **Error Recovery:** Gracefully handling errors without causing excessive requests

These measures ensure that the scraper operates responsibly and minimizes its impact on Myntra's servers.

18. Technology Used

18.1 Programming Language: Python 3.9+

18.1.1 Language Features Utilized

Python 3.9+ brings several advantageous features to this project:

- **Type Hints:** Improved static type checking capabilities
- **Dictionary Merging:** Enhanced dict operations with the `|` operator
- **String Methods:** New string methods for prefix/suffix handling
- **Improved Performance:** Faster startup and execution times
- **Better Error Messages:** More descriptive exception handling

18.1.2 Python for Web Scraping

Python is particularly well-suited for web scraping due to:

- **Rich Ecosystem:** Comprehensive libraries for HTTP, HTML parsing, and data processing
- **Expressive Syntax:** Clean, readable code that maps well to scraping workflows
- **Dynamic Typing:** Flexibility when handling varied website structures
- **Exception Handling:** Robust mechanisms for error management
- **Community Support:** Extensive documentation and examples for scraping tasks

18.2 Environment Management: Anaconda

18.2.1 Anaconda Distribution Benefits

Anaconda provides several advantages for this project:

- **Environment Isolation:** Prevents dependency conflicts between projects
- **Cross-Platform Support:** Works consistently across Windows, macOS, and Linux
- **Package Management:** Simplified installation of complex dependencies
- **Version Control:** Precise specification of package versions
- **Reproducibility:** Environment export/import for consistent deployments

18.2.2 Conda vs. Virtualenv

Anaconda was chosen over alternatives like virtualenv because:

- **Binary Package Distribution:** Pre-compiled packages avoid complex build dependencies
- **Non-Python Dependencies:** Better handling of C/C++ libraries and system dependencies
- **Environment Sharing:** Simplified sharing of complete environments via environment.yml
- **Multiple Python Versions:** Easier management of different Python interpreter versions
- **Integrated Tools:** Built-in support for Jupyter, Spyder, and other scientific tools

18.3 HTTP Client: Requests

18.3.1 Key Features Utilized

The Requests library provides essential capabilities:

- **Simple API:** Clean, intuitive interface for HTTP operations
- **Session Management:** Efficient handling of cookies and connection pooling
- **Header Customization:** Fine-grained control of HTTP headers
- **Status Handling:** Clear status code checking and exception raising
- **Content Decoding:** Automatic handling of response encoding

18.3.2 Advantages Over Alternatives

Requests was selected over alternatives because:

- **Simplicity:** More straightforward than urllib or http.client
- **Maturity:** Well-tested and stable across many use cases
- **Documentation:** Excellent documentation and examples
- **Community Support:** Widely used with abundant resources
- **Maintenance:** Active development and security updates

18.4 HTML Parser: BeautifulSoup4

18.4.1 Parser Capabilities

BeautifulSoup4 provides powerful HTML processing:

- **Forgiving Parsing:** Handles malformed HTML gracefully
- **Multiple Backends:** Support for different parser engines
- **Intuitive Navigation:** Simple methods for traversing the DOM
- **Flexible Selection:** Multiple ways to locate elements (tags, classes, CSS selectors)
- **Text Extraction:** Clean methods for getting text content

18.4.2 Selection Methods Used

The project leverages several BeautifulSoup selection techniques:

- **find() Method:** Locates the first matching element
- **find_all() Method:** Retrieves all matching elements
- **Class Selection:** Identifies elements by their CSS classes
- **Tag Selection:** Finds elements by their HTML tag names
- **Attribute Access:** Retrieves element attributes like “href” and “src”

18.5 Database: MongoDB

18.5.1 Document-Oriented Approach

MongoDB’s document model offers several benefits:

- **Schema Flexibility:** Accommodates varying product structures
- **JSON-Like Documents:** Natural representation of web data
- **Nested Data Support:** Effectively stores hierarchical information
- **Query Capabilities:** Powerful querying and aggregation
- **Indexing Options:** Multiple indexing strategies for performance

18.5.2 Local vs. Atlas Considerations

The project supports both deployment options:

Local MongoDB:

- Zero latency for development
- Complete control over configuration
- No usage limitations or costs
- Suitable for development and testing

MongoDB Atlas:

- Managed infrastructure with automated backups
- Simplified scaling and replication
- Global distribution options
- Better for production deployments

18.6 MongoDB Driver: PyMongo

18.6.1 Driver Features

PyMongo provides the interface between Python and MongoDB:

- **Native Data Type Conversion:** Automatically maps Python types to BSON
- **Connection Pooling:** Efficient management of database connections
- **Cursor Management:** Memory-efficient handling of query results
- **CRUD Operations:** Comprehensive support for all database operations
- **Aggregation Pipeline:** Support for complex data aggregation

18.6.2 Integration Patterns

The project implements several PyMongo patterns:

- **Client-DB-Collection Pattern:** Clear hierarchy of connection objects
- **Document Validation:** Checking required fields before insertion
- **Bulk Operations:** Using insert_many() for efficient batch processing
- **Error Handling:** Proper exception management for database operations
- **Connection Management:** Appropriate handling of database connections

18.7 Environment Variables: Dotenv

18.7.1 Configuration Management

Dotenv provides a clean approach to configuration:

- **Environment Separation:** Different settings for development vs. production
- **Security Benefits:** Keeping credentials out of source code
- **Simplicity:** Easy-to-understand .env file format
- **Framework Agnostic:** Works with any Python application
- **Local Overrides:** Support for machine-specific settings

18.7.2 Implementation Pattern

The project follows standard dotenv practices:

- **.env File:** Stores key-value pairs in a simple format
- **Early Loading:** Variables loaded at module import time
- **Default Values:** Fallbacks for missing environment variables
- **Type Conversion:** Converting string values to appropriate types
- **Configuration Grouping:** Related settings grouped together

18.8 User Agent Rotation: Fake-UserAgent

18.8.1 Anti-Detection Strategy

Fake-UserAgent helps avoid scraping detection:

- **Realistic Headers:** Generates current, authentic User-Agent strings
- **Browser Variety:** Rotates between different browser identities
- **Version Accuracy:** Includes appropriate version numbers
- **Platform Diversity:** Represents different operating systems
- **Automatic Updates:** Stays current with browser release cycles

18.8.2 Implementation Benefits

This technology provides several advantages:

- **Reduced Blocking:** Less likely to be identified as a bot

- **Request Distribution:** Appears as different browsers across requests
- **Normal Traffic Simulation:** Mimics regular user browsing patterns
- **Simple Integration:** Easy to incorporate into request headers
- **Maintenance-Free:** No need to manually update User-Agent strings

19. Result: Multi-Platform Review Scraping and Comparison

19.1 Scraper Capabilities

The completed Myntra Web Scraper successfully demonstrates these capabilities:

19.1.1 Data Extraction

The scraper effectively extracts product information from Myntra search results:

- **Search Term Processing:** Correctly handles search queries with proper URL encoding
- **Product Detection:** Accurately identifies product card elements in search results
- **Data Field Extraction:** Reliably extracts brand, name, price, and URL information
- **Image URL Retrieval:** Captures product image URLs for visual reference
- **Structure Preservation:** Maintains the relationship between product attributes

19.1.2 Data Processing

The extracted data undergoes appropriate transformation:

- **Text Normalization:** Removes extraneous whitespace and formatting
- **Price Formatting:** Converts price strings to numeric values
- **URL Standardization:** Ensures complete, absolute URLs for products and images
- **Type Conversion:** Applies appropriate data types to different fields
- **Error Handling:** Gracefully manages missing or malformed data

19.1 Multi-Platform Scraper Capabilities

19.1.1 Multi-Platform Data Extraction

- The scraper successfully extracts review data from three major e-commerce platforms:
- **Myntra:** Extracts product reviews, ratings, prices, and customer feedback from fashion/ lifestyle products
- **Flipkart:** Collects reviews across multiple product categories with robust parsing strategies
- **Amazon:** Handles complex review structures including verified purchases and detailed ratings
- Each platform scraper handles platform-specific HTML structures and JavaScript rendering

19.2 Sample Output Analysis

19.2.1 Comparison DataFrame Structure

The comparison output includes the following structure:

```
{
  • "Platform": "myntra",
  • "Product Name": "Nike Air Max Sneakers",
  • "Price": "₹7,499",
  • "Price_Numeric": 7499.0,
  • "Avg_Rating": 4.2,
  "Overall_Rating": 4.5,
  "Review_Count": 150
  • }
  •
  • This structure:
  • Enables price comparison across platforms (Price_Numeric column)
  • Provides rating metrics for informed decision-making
```

19.3 Performance Characteristics

19.3.1 Parallel Scraping Performance

The system demonstrates efficient multi-platform operation:

- Sequential Time: ~15-20 seconds per platform (3 platforms = 45-60 seconds)
- Parallel Time: ~15-20 seconds total (all 3 platforms simultaneously)
- Speed Improvement: Approximately 3x faster with parallel execution
- Resource Utilization: Efficient use of network bandwidth and CPU
- Scalability: Adding more platforms does not linearly increase total time

19.3.2 Extraction Success Rates

- For a typical search query like "men sneakers", the scraper achieves:
- Myntra: 90-95% success rate in extracting reviews
- Flipkart: 85-90% success rate with robust fallback parsing strategies
- Amazon: 80-90% success rate (varies due to bot detection measures)
- Overall Platform Coverage: Successfully extracts data from 2-3 platforms per search

19.4 Reliability Assessment

19.4.1 Multi-Platform Error Handling

The scraper demonstrates robust error management across platforms:

- Platform Independence: Failure of one platform does not stop others
- Retry Mechanisms: Automatic retries for network errors (3 attempts per request)
- Graceful Degradation: Comparison works even if only 1-2 platforms succeed
- Error Containment: Exceptions are caught and logged without crashing the application
- User Feedback: Clear error messages indicate which platforms failed

19.4.2 Edge Case Handling

- The implementation accommodates various edge cases:
- Missing Reviews: Continues processing when products have no reviews
- Price Format Variations: Handles different currency formats and missing prices
- Rating Variations: Processes both numeric and text-based ratings
- Platform-Specific Structures: Adapts to different HTML structures across platforms

19.5 Project Success Criteria

The project successfully meets its primary success criteria:

- Multi-Platform Extraction: Reliably extracts review data from Myntra, Flipkart, and Amazon
- Cross-Platform Comparison: Provides comprehensive comparison of prices, ratings, and reviews
- Modular Architecture: Maintains clean separation with BaseScraper pattern and service classes
- User Interface: Delivers intuitive Streamlit web interface for non-technical users
- Parallel Execution: Efficiently utilizes concurrent processing for multi-platform scraping

Data Storage: Properly stores structured data in MongoDB with platform organization

20. Future Work and Conclusion

20.1 Future Work

20.1.1 Pagination Support

To scrape multiple result pages:

- **Implementation Approach:** Add a loop to iterate through page numbers

- **URL Structure:** Modify URL construction to include pagination parameters
- **Result Aggregation:** Combine products from multiple pages
- **Progress Tracking:** Implement tracking of current page and total pages
- **Page Limit Controls:** Add configuration for maximum pages to scrape

```
def fetch_search_results_paged(query, max_pages=5):
    all_html = []
    for page in range(1, max_pages + 1):
        page_url = f"https://www.myntra.com/{query.replace(' ', '-')}"
        if page > 1:
            page_url += f"?p={page}"
        # Fetch page and add to results
        response = requests.get(page_url, headers={"User-Agent": UserAgent().
random})
        if response.status_code == 200:
            all_html.append(response.text)
        else:
            break # Stop if we get an error
        time.sleep(1) # Courteous delay between pages
    return all_html
```

20.1.2 Scheduled Execution

For regular data collection:

- **Cron Integration:** Set up periodic execution using cron jobs
- **Windows Task Scheduler:** Configure scheduled tasks on Windows
- **Airflow Workflows:** Implement more complex scheduling with Apache Airflow
- **Docker Containers:** Package the scraper for containerized deployment
- **Cloud Functions:** Deploy as serverless functions with scheduled triggers

Example cron entry for daily execution at 2 AM

```
0 2 * * * cd /path/to/scraper && conda run -n myntra_scraper python main.py
```

20.1.3 Data Visualization

To analyze collected data:

- **Pandas Integration:** Add data export to pandas DataFrames
- **Matplotlib Charts:** Create price distribution and trend visualizations
- **Interactive Dashboards:** Build web dashboards with Dash or Streamlit
- **Time Series Analysis:** Track price changes over time
- **Brand Comparisons:** Visualize price ranges by brand

```
def visualize_price_distribution(collection_name):
    # Get data from MongoDB
    client = MongoClient(os.getenv("MONGO_URI"))
    db = client[os.getenv("DB_NAME")]
    collection = db[collection_name]
```

Convert to DataFrame

```
df = pd.DataFrame(list(collection.find()))
```

```
# Create visualization
plt.figure(figsize=(10, 6))
df['price'].hist(bins=20)
plt.title('Price Distribution of Products')
plt.xlabel('Price (INR)')
plt.ylabel('Number of Products')
plt.savefig('price_distribution.png')
```

20.1.4 JavaScript-Rendered Content

To handle dynamic content:

- **Selenium Integration:** Add browser automation with Selenium
- **Playwright Support:** Use Playwright for modern browser automation
- **Headless Mode:** Run browsers in headless mode for efficiency
- **Wait Conditions:** Implement explicit waits for dynamic content
- **Event Handling:** Trigger JavaScript events to load additional content

```
def fetch_dynamic_content(url):
    from selenium import webdriver
    from selenium.webdriver.chrome.options import Options
    from selenium.webdriver.common.by import By
    from selenium.webdriver.support.ui import WebDriverWait
    from selenium.webdriver.support import expected_conditions as EC

    options = Options()
    options.headless = True
    driver = webdriver.Chrome(options=options)

    try:
        driver.get(url)
        # Wait for product cards to load
        WebDriverWait(driver, 10).until(
            EC.presence_of_all_elements_located((By.CLASS_NAME, "product-base
"))
        )
        return driver.page_source
    finally:
        driver.quit()
```

20.1.5 Advanced Features

For enhanced functionality:

- **Product Detail Scraping:** Extract detailed information from product pages
- **Category Navigation:** Traverse Myntra's category structure
- **Filter Support:** Apply filters to search results (e.g., price range, discount)
- **Review Extraction:** Collect customer reviews and ratings

- **Inventory Tracking:** Monitor product availability over time

```
def get_product_details(product_url):
    """Scrape detailed information from a product page."""
    response = requests.get(product_url, headers={"User-Agent": UserAgent().random})
    soup = BeautifulSoup(response.text, "html.parser")

    details = {}
    # Extract product details
    details["description"] = soup.select_one(".pdp-product-description").text.strip()
    details["specifications"] = [li.text.strip() for li in soup.select(".pdp-productDescriptors .pdp-product-descriptorstable")]
    details["size_chart"] = {size.text.strip(): size.get("class") for size in soup.select(".size-buttons-size-button")}
    details["rating"] = float(soup.select_one(".pdp-rating .pdp-rating-average").text) if soup.select_one(".pdp-rating .pdp-rating-average") else None
    details["review_count"] = int(soup.select_one(".pdp-rating-count").text.replace("Reviews", "").strip()) if soup.select_one(".pdp-rating-count") else 0

    return details
```

20.2 Conclusion

20.2.1 Project Summary

The Myntra Web Scraper project has successfully demonstrated:

- **Effective Data Extraction:** Reliable retrieval of product information from Myntra's search results
- **Modular Architecture:** Clean separation of concerns across distinct components
- **Database Integration:** Proper storage of structured data in MongoDB
- **Environment Management:** Reproducible setup using Anaconda
- **Ethical Implementation:** Responsible scraping with appropriate safeguards

This implementation serves as both a practical tool for data collection and an educational example of web scraping principles.

20.2.2 Technical Achievements

From a technical perspective, the project showcases:

- **Modern Python Practices:** Utilizing Python 3.9+ features and idioms
- **Error Resilience:** Robust handling of network and parsing failures
- **Clean Code Principles:** Readable, maintainable, and well-structured code
- **NoSQL Database Usage:** Effective use of MongoDB's document model
- **Configuration Management:** Proper separation of code and configuration

20.2.3 Educational Value

As an educational resource, the project provides insights into:

- **Web Scraping Techniques:** Practical approaches to HTML parsing and data extraction
- **HTTP Client Usage:** Proper implementation of web requests with headers and error handling
- **Database Design:** Schema considerations for scraped content
- **Environment Management:** Using Anaconda for reproducible development environments
- **Modular Software Design:** Applying separation of concerns and single responsibility principles

20.2.4 Practical Applications

The scraper enables several real-world applications:

- **Price Monitoring:** Tracking product prices over time for competitive analysis
- **Inventory Analysis:** Observing product availability patterns
- **Brand Research:** Comparing product offerings across different brands
- **Discount Tracking:** Identifying sales patterns and pricing strategies
- **Fashion Trend Analysis:** Monitoring product descriptions and categories for trend identification

20.2.5 Final Thoughts

This project demonstrates that effective web scraping solutions can be built with relatively simple components when properly designed. The modular architecture, MongoDB storage, and Anaconda environment create a flexible and extensible foundation that can grow to accommodate more complex requirements.

While the current implementation meets its core objectives, the outlined future enhancements provide a clear path for evolution into a more comprehensive data collection and analysis platform. By maintaining the principles of ethical scraping and clean design, these enhancements can be implemented without sacrificing maintainability or responsible operation.

21. References

21.1 Official Documentation

1. Python Documentation

- Python Language Reference:
<https://docs.python.org/3/reference/index.html>
- Python Standard Library: <https://docs.python.org/3/library/index.html>

2. Requests Library

- Official Documentation: <https://docs.python-requests.org/en/latest/>
- User Guide: <https://requests.readthedocs.io/en/latest/user/quickstart/>

3. BeautifulSoup4

- Documentation:
<https://www.crummy.com/software/BeautifulSoup/bs4/doc/>
- Quick Start:
<https://www.crummy.com/software/BeautifulSoup/bs4/doc/#quick-start>

4. MongoDB and PyMongo

- MongoDB Manual: <https://docs.mongodb.com/manual/>
- PyMongo Documentation: <https://pymongo.readthedocs.io/en/stable/>
- MongoDB Query Guide: <https://docs.mongodb.com/manual/tutorial/query-documents/>

5. Anaconda

- Anaconda Documentation: <https://docs.anaconda.com/>
- Conda User Guide: <https://docs.conda.io/projects/conda/en/latest/user-guide/index.html>

21.2 Web Scraping Resources

1. Ethical Scraping

- “Web Scraping Ethics” by Towards Data Science
- “Legal and Ethical Considerations in Web Scraping” - Research Paper by M. Krotov et al.
- robots.txt Protocol Documentation:
<https://www.robotstxt.org/robotstxt.html>

2. **Web Scraping Techniques**

- “Web Scraping with Python” by Ryan Mitchell
- “Mining the Social Web” by Matthew A. Russell
- “Automate the Boring Stuff with Python” by Al Sweigart

3. **HTML Parsing**

- HTML5 Specification: <https://html.spec.whatwg.org/>
- CSS Selectors Reference: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Selectors

4. **HTTP Protocol**

- HTTP/1.1 Specification: <https://tools.ietf.org/html/rfc7231>
- MDN HTTP Documentation: <https://developer.mozilla.org/en-US/docs/Web/HTTP>

21.3 E-commerce Analysis

1. **E-commerce Data Mining**

- “Data Mining in E-commerce: Past, Present and Future” by Ngai et al.
- “Market Basket Analysis: A Review” by Kaur et al.

2. **Fashion Retail Analytics**

- “Fashion Retail: The Journey from Data to Decisions” - Industry Report
- “Predictive Analytics in Fashion Retail” - Research Paper

3. **Price Monitoring**

- “Dynamic Pricing in E-commerce” - Research Paper
- “Competitive Price Intelligence in Retail” - Industry Report

4. **Trend Analysis**

- “Fashion Trend Forecasting” - Academic Research
- “Predictive Analytics for Fashion” by Neri Karra

21.4 Project-Specific Resources

1. **Myntra Website**

- Myntra Homepage: <https://www.myntra.com>
- Myntra Developer Resources (if available)

2. **Python Libraries**

- fake-useragent documentation: <https://pypi.org/project/fake-useragent/>
- python-dotenv documentation: <https://pypi.org/project/python-dotenv/>

3. **NoSQL Database Design**

- “NoSQL Database Design for MongoDB” - Best Practices
- “Schema Design for E-commerce” - MongoDB White Paper

4. **Software Architecture**

- “Clean Architecture” by Robert C. Martin
- “Python Application Layouts: A Reference” by Real Python

5. **Web APIs and Microservices**

- “RESTful Web APIs” by Leonard Richardson et al.
- “Building Microservices” by Sam Newman

3.9.1 Browser Automation Capabilities

Selenium WebDriver provides essential browser automation for dynamic content:

Headless Browser Support: Can run Chrome in headless mode for efficiency

JavaScript Execution: Handles dynamically rendered content that Requests cannot access

Element Interaction: Can click buttons, scroll pages, and wait for content to load

Cross-Platform: Works consistently across Windows, Linux, and macOS

Multiple Browser Support: Supports Chrome, Firefox, Edge, and Safari

3.9.2 Implementation in Our Project

The project uses Selenium WebDriver for:

Navigating search result pages and product pages across all three platforms

Scrolling through review pages to load lazy-loaded content

Waiting for dynamic content to appear before extraction

Handling popups, cookies, and other interactive elements

Executing JavaScript to trigger content loading

3.9.3 Advantages Over Static Scraping

Selenium enables scraping of modern e-commerce sites that:

Load content via JavaScript after initial page load

Require user interactions to reveal additional content

Use infinite scroll or lazy loading for reviews

Implement bot detection that requires realistic browser behavior

3.10.1 Web Application Framework Features

Streamlit provides a Python-based framework for building web applications:

Rapid Development: Create interactive web apps with pure Python

No Frontend Knowledge Required: Build UIs using simple Python functions

Built-in Components: Pre-built widgets for inputs, charts, and displays

Session State Management: Maintains user state across interactions

Automatic Reruns: Updates UI automatically when inputs change

3.10.2 Implementation in Our Project

The app.py module uses Streamlit to provide:

Mode Selection: Radio buttons for Single Platform vs Compare All Platforms

Product Search Input: Text input for search queries

Platform Selection: Dropdown for choosing individual platforms

Result Display: DataFrames, charts, and comparison metrics

Interactive Visualizations: Plotly charts for price and rating comparisons

Status Indicators: Spinners and success/error messages

3.10.3 User Experience Benefits

Streamlit enables a user-friendly interface that:

Requires no technical knowledge to operate

Provides immediate visual feedback

Displays results in an organized, readable format

Supports interactive exploration of scraped data

3.11.1 Parallel Processing Capabilities

ThreadPoolExecutor (from concurrent.futures) enables parallel execution:

Concurrent Scraping: Run multiple platform scrapers simultaneously

Thread Pool Management: Efficiently manages worker threads

Future Objects: Track completion status of parallel operations

Resource Efficiency: Maximizes CPU and network utilization

Graceful Error Handling: Individual thread failures don't crash the entire operation

3.11.2 Implementation in ComparisonService

The ComparisonService class uses ThreadPoolExecutor to:

Scrape Myntra, Flipkart, and Amazon concurrently

Reduce total scraping time from sequential sum to maximum individual time

Handle platform failures independently (one platform failure doesn't stop others)

Collect results as they complete via `as_completed()`

3.11.3 Performance Benefits

Parallel execution provides significant advantages:

Time Reduction: 3x faster when scraping all platforms (3 scrapers run simultaneously)

Better User Experience: Results appear faster, improving perceived performance

Scalability: Easy to add more platforms without linear time increase

Resource Utilization: Better use of available network bandwidth

3. Scraper executes the workflow: `scrape_product_urls()` → `extract_reviews()` → `extract_products()`

4. Results are stored in MongoDB with platform identifier

5. DataFrame is displayed in Streamlit interface

7.1.3 Multi-Platform Comparison Mode Workflow

When user selects Compare All Platforms mode:

1. User enters product name and number of products per platform

2. ComparisonService is instantiated with product name

3. `scrape_all_platforms()` method uses ThreadPoolExecutor to run all three scrapers in parallel

4. Each scraper runs independently and returns a DataFrame of reviews

5. ComparisonService generates comparison DataFrame with metrics (prices, ratings, review counts)

6. Results are displayed with interactive charts showing price and rating comparisons
7. All reviews are combined and stored in MongoDB with platform tags

```
@abstractmethod
```

```
def extract_products(self, product_reviews) -> pd.DataFrame:
```

```
    """Extract individual reviews from review pages"""
```

```
    pass
```

```
def get_review_data(self) -> pd.DataFrame:
```

```
    """Orchestrator method - common workflow for all platforms"""
```

```
    # 1. Scrape product URLs
```

```
    # 2. For each product, extract reviews
```

```
    # 3. Combine all reviews
```

```
    # 4. Add Platform column
```

```
    # 5. Return DataFrame
```

7.2.2 Platform-Specific Implementations

Each platform scraper inherits from BaseScraper and implements abstract methods:

Myntra (ScrapeReviews):

- scrape_product_urls(): Searches Myntra using hyphenated URL format
- extract_reviews(): Finds review link on product page
- extract_products(): Parses Myntra review page structure

Flipkart (FlipkartScraper):

- scrape_product_urls(): Searches Flipkart and extracts /p/ product URLs
- extract_reviews(): Constructs /product-reviews/ URL from product page
- extract_products(): Parses Flipkart review containers with multiple fallback strategies

Amazon (AmazonScraper):

- `scrape_product_urls()`: Searches Amazon and extracts /dp/ product URLs
- `extract_reviews()`: Navigates to product page, scrolls to reviews section
- `extract_products()`: Extracts reviews from product page (reviews on same page)

7.2.3 Common Functionality

BaseScraper provides shared methods used by all platforms:

- `scroll_to_load_reviews()`: Infinite scroll implementation
- `get_review_data()`: Main orchestrator with error handling
- WebDriver setup with anti-detection measures
- Consistent DataFrame structure (Product Name, Over_All_Rating, Price, Date, Rating, Name, Comment, Platform)

7.3.1 Parallel Scraping Architecture

The ComparisonService class orchestrates multi-platform scraping:

```
from concurrent.futures import ThreadPoolExecutor, as_completed
```

```
class ComparisonService:
```

```
    def scrape_all_platforms(self):
```

```
        platforms = ["myntra", "flipkart", "amazon"]
```

```
        with ThreadPoolExecutor(max_workers=3) as executor:
```

```
            futures = {executor.submit(self._run_scraper, p): p for p in platforms}
```

```
            for future in as_completed(futures):
```

```
                platform, data = future.result()
```

```
                if data is not None:
```

```
                    results[platform] = data
```

```
return results
```

7.3.2 Comparison Generation

After collecting data from all platforms, the service generates comparisons:

```
def generate_comparison(self, platform_data):  
    # For each platform and product:  
    # - Normalize prices using price_normalizer  
    # - Calculate average ratings  
    # - Count reviews  
    # - Extract overall ratings  
    # Returns DataFrame with: Platform, Product Name, Price, Price_Numeric,  
    #                         Avg_Rating, Overall_Rating, Review_Count
```

7.3.3 Summary Statistics

`get_comparison_summary()` generates key metrics:

- Total platforms found
- Total products
- Total reviews
- Cheapest platform and price
- Best rated platform and rating

7.3.4 Price Normalization

The `price_normalizer` utility handles different price formats:

- Removes currency symbols (₹, Rs., INR)
- Removes commas
- Extracts numeric values

- Creates Price_Numeric column for calculations
- Handles missing/NA values gracefully

19.1.2 Review Data Extraction

The system effectively extracts comprehensive review information:

Product Names: Accurate product identification across platforms

Prices: Normalized price extraction handling various currency formats

Overall Ratings: Product-level rating scores

Individual Review Ratings: Star ratings for each customer review

Review Comments: Full text of customer reviews and feedback

Reviewer Names: Customer identification (when available)

Review Dates: Timestamp information for temporal analysis

Platform Tags: Automatic platform identification for each review

19.1.3 Cross-Platform Comparison

The ComparisonService generates comprehensive comparative analysis:

Price Comparison: Identifies cheapest platform for each product

Rating Comparison: Highlights best-rated products across platforms

Review Count Analysis: Shows review volume per platform

Visual Charts: Interactive Plotly charts for price and rating distributions

Summary Metrics: Best deals, best ratings, and platform coverage statistics

Shows review volume for credibility assessment

Maintains platform identification for data provenance

Supports filtering and sorting operations

19.2.2 Review DataFrame Structure

Individual reviews are stored with consistent schema:

```
{  
  "Product Name": "Nike Air Max Sneakers",  
  "Over_All_Rating": "4.5",  
  "Price": "₹7,499",  
  "Date": "2 months ago",  
  "Rating": "5",  
  "Name": "John Doe",  
  "Comment": "Great quality and comfort...",  
  "Platform": "myntra"  
}
```

This enables:

Sentiment analysis across platforms

Temporal analysis of reviews

Reviewer behavior comparison

Platform-specific review pattern identification

Review Extraction: Typically 10-50 reviews per product depending on platform

19.3.3 Data Quality Metrics

The extracted data demonstrates high quality:

Price Normalization: 95%+ success rate in converting various formats to numeric

Rating Extraction: 90%+ success rate in extracting valid ratings

Review Text Completeness: 85%+ reviews have substantial comment text

Platform Tagging: 100% accuracy in platform identification

Schema Consistency: All platforms produce DataFrames with identical structure

Dynamic Content Loading: Waits for JavaScript-rendered content before extraction

Bot Detection: Implements anti-detection measures (user agents, realistic delays)

Error Resilience: Handles platform failures gracefully with independent error handling

Ethical Operation: Implements reasonable measures to respect website terms of service

These results demonstrate that the scraper fulfills its intended purpose as both a practical tool for

multi-platform product comparison and an educational example of advanced web scraping techniques

including browser automation, parallel processing, and data comparison workflows.